

INGEGNERIA DELLA CONOSCENZA

Alessandro Carella

INDICE

1. Ingegneria della conoscenza	4
1.1. Sistemi intelligenti basati su Conoscenza	4
1.2. Dimensioni dello spazio di progettazione	5
1.2.1. Modularità	6
1.2.2. Schema di Rappresentazione	6
1.3. Progettazione si Sistemi Intelligenti	11
1.3.1. Semplificazione di Ambienti e Sistemi	11
1.3.2. Compiti e Problemi	11
1.3.3. Rappresentazione interna	11
1.4. Soluzioni	12
1.4.1. Modelli: Livelli di Astrazione	12
1.5. Sistemi Intelligenti basati su Conoscenza	13
1.6. Domini applicativi	14
2. Ricerca di soluzioni in spazi di stati	14
2.1. Risoluzione di problemi mediante ricerca	14
2.2. Spazi di stati	15
2.3. Ricerca su Grafo	17
2.4. Algoritmo di ricerca generico	18
2.5. Strategie di ricerca non informate	20
2.5.1. Ricerca in ampiezza	20
2.5.2. Ricerca in profondità	21
2.5.3. Approfondimento iterativo	22
2.5.4. Ricerca di soluzioni a costo minimo	23
2.6. Ricerca Informata (Euristica)	24
2.6.1. Euristica	24
2.6.2. DFS Euristica e Greedy best-First Search	25
2.6.3. Ricerca A^*	25
2.6.4. Algoritmo Branch and Bound	27
2.6.5. Progettazione della funzione Euristica	29
2.7. Potatura dello Spazio di Ricerca	29
2.7.1. Potatura dei Cicli	30
2.7.2. Potatura di Percorsi Multipli	30
2.7.3. Consuntiva sulle Strategie di Ricerca	32
2.8. Strategia di Ricerca più sofisticate	33
2.8.1. Direzione della ricerca	33
2.8.2. Ricerca bidirezionale	33

2.8.3. Ricerca in una gerarchia di astrazioni	33
2.8.4. Uso della programmazione dinamica	34
3. Ragionamento con vincoli	35
3.1. Variabili e vincoli	35
3.1.1. Variabili e Assegnazioni	35
3.1.2. Vincoli	36
3.1.3. Problema di soddisfacimento di vincoli	37
3.2. Risoluzione di CSP tramite ricerca	38
3.2.1. Algoritmi su Grafo di Ricerca	38
3.3. Algoritmi basati su Consistenza	38
3.4. Separazione dei domini	39
3.5. Eliminazione di Variabili	40
3.6. Ricerca Locale	42
3.6.1. Miglioramento Iterativo	43
3.6.2. Algoritmi stocastici	44
3.7. Varianti della ricerca Locale	45
3.7.1. Tabu Search	45
3.7.2. Passo di Massimo Miglioramento	45
3.7.3. Scelta a due fasi	46
3.7.4. Algoritmo Any-Conflict	46
3.7.5. Simulated Annealing	46

INGEGNERIA DELLA CONOSCENZA

Definizione 1. (Ingegneria della conoscenza) L'Ingegneria della Conoscenza è il processo di acquisizione, strutturazione e rappresentazione delle competenze umane in un formato che le macchine possono comprendere e utilizzare. Può essere vista come l'arte e la scienza di costruire sistemi basati sulla conoscenza in grado di ragionare, prendere decisioni e risolvere problemi complessi in modi che rispecchiano il pensiero umano.

Nel contesto dell'*intelligenza artificiale* che sta rimodellando ogni settore, l'ingegneria della conoscenza rappresenta il ponte fondamentale tra le competenze umane e l'intelligenza artificiale. Gli ingegneri della conoscenza possono essere considerati come traduttori della conoscenza implicita che *vive* nella mente degli esperti: essi la convertono in formati espliciti e strutturati che le macchine possono elaborare e sulle quali possono agire.

1.1. Sistemi intelligenti basati su Conoscenza

L'ingegneria della conoscenza è una disciplina ingegneristica atta a progettare e sviluppare *sistemi intelligenti* basati su conoscenza *esplicita*.

L'intelligenza artificiale può essere definita come la disciplina che studia sintesi e analisi degli *agenti computazionali*, ossia dei sistemi che agiscono in modo *intelligente*. Essi possono svolgere azioni in un *ambiente* e vanno giudicati **solo** per tali azioni.

Aspetti salienti dell'*azione intelligente* sono i seguenti:

- l'appropriatezza rispetto alle circostanze, agli obiettivi, ai limiti;
- la considerazione delle conseguenze a breve/lungo termine delle azioni;
- l'apprendimento dall'esperienza;
- la flessibilità rispetto ai cambiamenti.

Un **agente computazionale** è un agente le cui decisioni/azioni siano spiegabili in termini di calcolo automatico. Esso è implementato su sistema *fisico* e presenta le seguenti *limitazioni*:

- non è onnisciente né onnipotente;
- le capacità di osservazione sono specializzate e vincolate dal dominio applicativo;
- la sua memoria è finita e a tempo limitato.

La disciplina dell'AI può essere inquadrata sotto diverse prospettive:

- la prospettiva *scientifica* considera l'AI come disciplina finalizzata a comprendere i principi del comportamento intelligente:
 - analizzando sistemi naturali e artificiali;
 - formulando *teorie* sulla costruzione di agenti intelligenti che possano essere supportate da *implementazioni* per la loro *verifica sperimentale*;
- la prospettiva *ingegneristica* inquadra l'AI come lo studio di nuove tecnologie atte risolvere specifici problemi tipicamente risolti da agenti umani:
 - progetto e sintesi di *sistemi intelligenti basati su conoscenza* (KBS) *testabili*: qualità valutabili attraverso metodi standard dell'ing. informatica;
 - applicazioni costruite per la loro *utilità* in un dato dominio.

Altri obiettivi dell'AI:

- **intelligence augmentation**: accrescimento dell'intelligenza e della creatività umane. Alcuni esempi:
 - sistema diagnostico: aiuta i medici a prendere decisioni migliori;
 - motore di ricerca: aiuta a superare i limiti della memoria umana;
 - traduzione automatica: aiuta la gente a comunicare.
- **integrazione** dei compiti di agenti umani e sistemi intelligenti che collaborano alla soluzione di problemi; si parla di modalità **human-in-the-loop**.

C'è differenza tra la definizione di intelligenza *naturale* e l'intelligenza *artificiale*. L'intelligenza *naturale* emerge spontaneamente in natura mentre, quella *artificiale*, viene progettata per specifici scopi.

L'ipotesi fondamentale può essere riassunta con la seguente formula:

$$\text{ragionamento} \approx \text{computazione}$$

Tale ipotesi è collegata alla *tesi di Church-Turing*: esiste un livello di astrazione nel quale il ragionamento coincide con la manipolazione di simboli.

In termini generali potremmo definire la **conoscenza** un'informazione stabile, non transitoria.

1.2. Dimensioni dello spazio di progettazione

La *complessità* dei sistemi intelligenti varia e dipende dalle finalità e dai diversi contesti in cui sono immersi. Ciò vale dai casi semplici a quelli più complessi.

Le **dimensioni della complessità** nella progettazione:

- definiscono uno *spazio di progettazione*: si hanno diversi sistemi a seconda delle loro tipologie;
- forniscono forme di *decomposizione sommaria* dello spazio, cui andranno aggiunte altre scelte da fare;
- interagiscono fra loro: dimensioni combinabili anche se studiate separatamente.

Dimensione	Valori
Modularità	piatta modulare gerarchica
Orizzonte della pianificazione	nessuna pianificazione #tappe finito (miopia) indefinito infinito (processo)
Rappresentazione del mondo	stati (tuple di) feature individui + relazioni (binarie) #mondi: n_s 2^{n_f} $2^{n_i} \cdot n_r$ (n_s = #stati, n_f = #feature, n_i = #individui e n_r = #relazioni)
Incertezza	<i>osservabilità del mondo</i> : totale parziale <i>effetto</i> : deterministico stocastico
Preferenze	obiettivi (da raggiungere mantenere) compromessi (preferenze cardinali ordinali)
Apprendimento	<i>conoscenza data</i> / <i>appresa</i>
Limiti delle risorse	<i>razionalità perfetta</i> / <i>limitata</i>
Numero di attori	singolo multipli agenti (distribuiti, cooperativi competitivi)
Interattività	<i>ragionamento (computazione)</i> online / offline

1.2.1. Modularità

Il tipo di **modularità** indica il grado di decomposizione di un sistema in *moduli* interagenti da prendere in considerazione separatamente. Ciò serve a *dominare la complessità*, nei sistemi software ma anche nelle organizzazioni. Tipicamente la modularità viene espressa attraverso una decomposizione gerarchica: ogni modulo è organizzato in sotto-moduli a loro volta organizzati gerarchicamente fino al livello base; astrazione procedurale e OOP servono a sfruttare modularità e astrazione.

Le strutture possibili sono: *piatta*, ossia senza struttura organizzativa; *modulare*, se il sistema viene decomposto in moduli interagenti considerabili separatamente; *gerarchica* quando nel sistema i moduli possono essere decomposti in sotto-moduli interagenti, a loro volta decomponibili.

Le modalità di *ragionamento* dipendono dal tipo di struttura: con una struttura piatta o modulare si ha un singolo livello di astrazione; con una gerarchica si hanno multipli livelli di astrazione.

1.2.2. Schema di Rappresentazione

Lo **schema di rappresentazione** riguarda la *descrizione del mondo*. Le rappresentazioni alternative possibili sono: a *stati*, con *caratteristiche* / *proposizioni* oppure con individui e *relazioni*.

Stati

Gli **stati** distinti del mondo hanno un impatto sul comportamento del sistema. Essi sono fattorizzabili in stati *interni* e stati dell'*ambiente*.

Esempio 1. Un termostato può essere modellato usando 6 stati:

- stati *interni*: **spento**, **riscaldamento**,
- stati *ambiente*: **freddo**, **confortevole**, **caldo**;
 - ambiente freddo: passare o restare in modalità riscaldamento;
 - ambiente molto caldo: può passare nello stato spento;
 - ambiente confortevole: rimanere nello stato corrente;
- *azioni*: scaldare nello stato riscaldamento, altrimenti passare nello stato spento.

Caratteristiche

Il ragionamento si basa su **caratteristiche**, dette *feature*, degli stati o *proposizioni* booleane invece della loro enumerazione. Uno stato può essere descritto in termini di caratteristiche, con un valore per ogni stato.

Esempio 2. (Domotica) In un sistema per la domotica le caratteristiche potrebbero essere:

- posizione degli interruttori;
- stato di ogni interruttore (*in_funzione*, *in_corto*, *fuori_uso*);
- stato dei punti luce, ad esempio *pos_s2* con valore *up* se l'interruttore *s2* è acceso e *down* se spento;
- stato della casa descritto in termini dei valori di ciascuna delle caratteristiche.

Proposizioni

Una **proposizione** è una caratteristica *booleana*. Risulta una rappresentazione *compatta*: sono state comprese *regolarità* importanti sul dominio. Ad esempio con 30 proposizioni codificano $2^{30} \approx 10^9$ stati quindi è più facile specificare e ragionare con 30 proposizioni che con oltre un miliardo di stati.

Relazioni e individui

Nella descrizione di mondi complessi una relazione su un singolo individuo è una **proprietà**. Si può definire una *caratteristica* per ogni possibile relazione tra individui, come ad esempio $R(a, b)$, che potrebbe corrispondere a «Nico studia Informatica», ossia $studia(Nico, Informatica)$. Le *descrizioni relazionali* sono più convenienti di caratteristiche e proposizioni:

- ad esempio, con una sola relazione binaria e 100 individui si possono rappresentare $100^2 = 10000$ proposizioni e 2^{10000} stati;
- si possono considerare intere classi di individui senza enumerarne caratteristiche/proposizioni, o addirittura i numerosissimi stati;

- permettono di ragionare su infiniti individui; ad esempio l'insieme dei numeri, o l'insieme di tutte le stringhe, cosa impossibile in termini di stati.

Incertezza

Si progetta anche tenendo conto dell'**incertezza** insita nel dominio considerato:

1. percezione / osservazione;
2. effetti delle decisioni / azioni.

Osservazione incerta

A volte è possibile l'*osservazione diretta* dello stato del mondo. Più spesso accade che la percezione dello stato sia *difettosa* o *parziale* / *indiretta*: al più si può disporre di una *distribuzione* di probabilità sull'insieme degli stati possibili su quanto si osserva.

L'**incertezza sulla percezione** riguarda la possibilità di determinare lo stato del mondo attraverso osservazioni. Uno stato può essere:

- *pienamente osservabile*: si può riconoscere dalle osservazioni, assunzione spesso fatta per ragioni di trattabilità dei problemi;
- *parzialmente osservabile*: non osservato direttamente; è possibile che più stati portino alle stesse osservazioni oppure le osservazioni sono *rumorose*.

Effetto incerto

In certi casi è possibile conoscere sempre l'effetto delle azioni/decisioni: dato uno stato e un'azione/decisione, si può predire *precisamente* lo stato risultante dall'applicazione dell'azione. A volte è difficile fare tale previsioni: al più si avere una *distribuzione di probabilità* sugli effetti possibili.

L'*incertezza sugli effetti* prevede che la loro *dinamica* possa essere:

- *deterministica* e lo stato risultante viene determinato esattamente dall'azione e dallo stato precedente;
- *aleatoria*, con una probabilità sui possibili stati risultanti: ha senso solo se il mondo è completamente osservabile.

Preferenza

Gli agenti sono spesso *utilitaristici*: la scelta di un'azione è dettata dai risultati attesi più desiderabili; ci possono essere le finalità *semplici* o anche preferenze *complesse*.

Le **preferenze** si caratterizzano come:

- *finalità*, da raggiungere, detto *achievement goal*, o di conservazione, detto *maintenance goal*, in ogni stato visitato;
- *preferenze complesse*: sono compromessi sul vantaggio derivante dei vari risultati, eventualmente anche in momenti diversi. Si distinguono:
 - preferenze *ordinali*: conta solo l'ordine;
 - preferenze *cardinali*: conta anche la grandezza del valore.

Apprendimento

Non sempre il progettista dispone di un buon modello del sistema e del suo ambiente: si devono usare dati da esperienze passate e altre sorgenti di conoscenza per migliorare il modello e prendere migliori decisioni.

La dimensione dell'**apprendimento** determina se la conoscenza sia data oppure se vada appresa.

Apprendere significa trovare il modello migliore che si adatti ai dati e si possono presentare diversi casi:

- caso *semplice*: regolare un insieme fisso di *parametri*;
- caso più difficile: scegliere preliminarmente la migliore *rappresentazione*, ad esempio feature o relazioni.

Si possono presentare diverse problematiche:

- utilizzo di *conoscenza di fondo*;
- *selezione* dei dati da raccogliere;
- *rappresentazione* dei dati e dei modelli;
- selezione dei *learning bias* appropriati;
- uso della conoscenza appresa per modificare decisioni / azioni dell'agente.

Limitatezza delle Risorse Computazionali

Le **limitazioni** sulle risorse computazionali disponibili spesso impediscono di prendere le migliori decisioni. Si fanno compromessi quindi sulla qualità della soluzione da cercare spesso necessari: spesso è preferibile una soluzione ragionevole ma rapida che una migliore calcolata quando è troppo tardi perché il mondo esterno nel frattempo è cambiato.

Tale dimensione determina se il sistema ragiona per prender la migliore decisione: senza tener conto dei limiti, *razionalità perfetta*, oppure tenendo conto dei limiti, *razionalità limitata*.

I *limiti* riguardano il *tempo*, la *memoria* e la *precisione*, il grado di approssimazione.

Un **algoritmo anytime** produce soluzioni che migliorano con il tempo:

- in qualunque momento produce la *miglior soluzione corrente*;
- si assicura che la qualità non decresca, si conserva la migliore soluzione trovata da restituire su richiesta;
- l'attesa può avere un *costo*: a volte meglio decidere/agire subito anziché di aspettare una soluzione probabilmente migliore.

Orizzonte

L'**orizzonte** misura quanto lontano sia prevista la pianificazione del lavoro, ossia quanto in avanti ci si spinga a considerare le conseguenze delle azioni. Si possono distinguere sistemi/agenti software:

- *senza pianificazione*: nelle decisioni non considerano il futuro e il fattore-tempo non è preso in considerazione;

- a *orizzonte finito*: interessano solo un numero prefissato di passi;
- a *orizzonte indefinito*: numero di passi finito, ma non predeterminato;
- a *orizzonte infinito*: sempre attivo.

Numero di agenti

Difficoltà aggiuntive degli ambienti con altri agenti / sistemi sono:

- occorre ragionare sugli altri agenti secondo *strategie*: potrebbero cercare di confondere e manipolare o potrebbero cooperare; conviene agire in modo casuale se gli altri adottano strategie deterministiche;
- anche quando si cooperi e vi sia un file comune, il problema della *coordinazione* e della *comunicazione* rende il ragionamento multi-agente più complesso.

Dal punto di vista del singolo agente, la dimensione del **numero di agenti** prevede:

- ragionamento da *agente singolo*: gli altri come parte dell'ambiente;
- ragionevole se non ci sono altri agenti o se gli altri non cambieranno il comportamento in base alle sue azioni;
- ragionamento *multi-agente*: si prende in considerazione il ragionamento altrui;
- in caso di agenti intelligenti con *fini* / *preferenze* che dipendano da quello dalla *comunicazione* con gli altri;
- più difficile se gli agenti possono agire *simultaneamente* o se l'ambiente è solo *parzialmente osservabile*.

Interazione delle Dimensioni

Non si possono studiare le dimensioni separatamente perchè sono soggette a *interazioni complesse*:

- *Rappresentazione e modularità*: avendo una gerarchia di moduli semplici, il ragionamento può essere svolto su insieme finito di stati; altri livelli di astrazione richiedono il ragionamento su relazioni.
- *Orizzonte e modularità*: ad esempio, un cane robotico può ricevere una ricompensa quando risponde al richiamo; quando decide sui singoli movimenti, il momento del premio potrebbe essere lontano quindi l'orizzonte risulta potenzialmente indefinito.
- *Incertezza sull'osservazione e complessità del ragionamento*: molto più facile ragionare quando si conosce lo stato del mondo; l'incertezza su individui e relazioni è più complicata da trattare.
- *Incertezza sugli effetti e modularità*: ad un dato livello della gerarchia, una decisione può essere *deterministica* mentre ad un altro potrebbe essere *stocastica*.
- *Molteplicità e modularità*: agente progettato attraverso più sistemi che interagiscono condividendo il *fine comune* di rendere intelligente il comportamento dell'agente al livello superiore.
- *Apprendimento e rappresentazione*: spesso l'apprendimento lavora sulla rappresentazione di feature: determina modelli che portano a migliori predizioni dei

valori di una specifica feature-obiettivi; ma si può lavorare anche su individui e relazioni, ovvero l'apprendimento di gerarchie di modelli, in domini parzialmente osservabili e attraverso sistemi multipli.

- *Modularità e razionalità limitata* per un ragionamento più efficiente; il formalismo può diventare più complicato ma per costruire sistemi complessi servono: la decomposizione in componenti più piccole e delle approssimazioni per poter decidere in tempo accettabili anche in regime di memoria limitata.

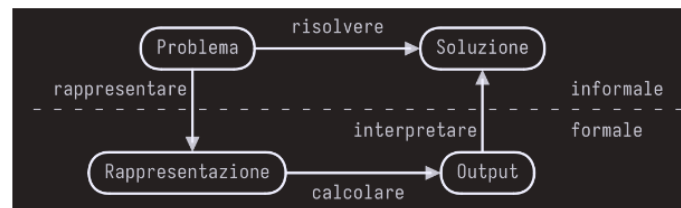


Figura 1.1: Risoluzione dei problemi

Figura 1: Risoluzione dei problemi

1.3. Progettazione di Sistemi Intelligenti

1.3.1. Semplificazione di Ambienti e Sistemi

La conoscenza del KBS non corrisponde a quella nella mente dell'esperto e/o del progettista.

Vi sono due casi-limite possibili:

- *sistema specializzato* nel suo dominio/task, poco utile fuori dal suo contesto;
- *sistema flessibile* che si adatta ai contesti e accetta nuovi task.

Le strategie di costruzioni alternative sono:

- creazione di un modello dell'ambiente *semplificato* utile a costruire sistemi di ragionamento *complessi*
- sistemi *semplici* per contesti *complessi*.

1.3.2. Compiti e Problemi

In AI è importante **cosa** vada fatto/calcolato e non **come** ridotto a ricerca nello spazio di soluzioni possibili. Una descrizione di un **problema** dev'essere disponibile a livello informale.

1.3.3. Rappresentazione interna

La **conoscenza** può essere definita come informazione *stabile*, a *lungo termine* su un dominio, a differenza delle **credenze** sull'ambiente che sono più *transitorie*. Per poterci ragionare essa va rappresentata formalizzandola in termini di un **linguaggio di rappresentazione**. Una **base di conoscenza** risulterà come rappresentazione

interna al sistema, codificata attraverso idonee strutture dati.

Le *proprietà* principali degli schemi di rappresentazione sono le seguenti:

- *ricchezza espressiva* sufficiente alla risoluzione del problema;
- *vicinanza* ai termini naturali del problema; esiste una relazione fra il dominio e la sua rappresentazione il che permette verifiche di correttezza; la vicinanza è anche correlata alla *spiegabilità* dei meccanismi di ragionamento interni;
- *trattabilità*, propensione a un'elaborazione efficiente;
- *acquisibilità* a partire da dati e/o da esperienza pregressa da parte degli utenti.

1.4. Soluzioni

Come nell'ingegneria del software, dato un problema, il progettista deve definire cosa costituisca una **soluzione**. Le soluzioni possono essere di diverso tipo:

- *ottimali*: la migliore secondo una *misura di qualità*:
 - *ordinale*, tipica;
 - *cardinale*, se contano anche le grandezze relative;
 - combinazione di criteri multipli.
- *soddisfacente*: buona secondo una specifica delle risposte *adeguate*;
- *approssimata*: soluzione di qualità *prossima* a quella ottimale, sulla base di una misura *cardinale*; conveniente per ragioni di efficienza anche in virtù degli algoritmi adottati;
- *probabile*: *verosimilmente* una soluzione, con un determinato grado di certezza:
 - forma di approssimazione precisa di soluzione soddisfacente;
 - si possono distinguere i tassi di errori di *falsi-positivi* o di *falsi-negativi*.

1.4.1. Modelli: Livelli di Astrazione

Un **modello** è una rappresentazione del mondo che può essere statica, ciò che si crede vero, ovvero dinamica, il suo *funzionamento*. È utile lavorare a vari livelli di dettaglio, ossia di **astrazione**: si rappresenta la parte del mondo di interesse trascurando i dettagli inutili. Il **livello di astrazione** determina un *ordine parziale*. Sono spesso ammissibili *più modelli* a diversi livelli di astrazione, anche in contraddizione tra loro: andranno giudicati in base alla loro utilità che per la loro correttezza.

Livelli d'astrazione comuni fra sistemi *biologici* e *computazionali* sono i seguenti:

- **livello della conoscenza** sul mondo esterno: considera quello che il sistema assume di sapere e i suoi obiettivi, ma non come ragiona, ossia non come calcolare la soluzione;
- **livello simbolico** interno: descrizione del modo di ragionare del sistema utile a implementare il livello precedente, manipolando simboli per produrre risposte.

1.5. Sistemi Intelligenti basati su Conoscenza

Un **agente intelligente** è un modello capace di percezione - ragionamento - azione che si trova immerso in un **ambiente**. Un possibile schema è il seguente:

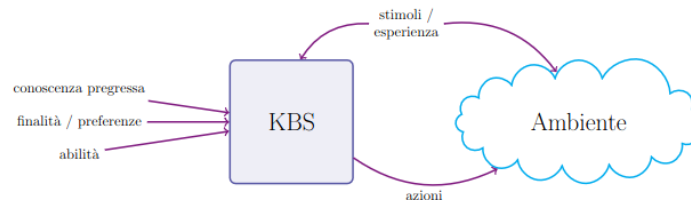


Figura 1.2: Schema di KBS / agente

Figura 2: Schema di un KBS

Il comportamento del KBS si basa su:

- **conoscenza pregressa** riguardo l'agente e il suo ambiente;
- **storia** dell'interazione con l'ambiente: **stimoli** ed **esperienza passata**;
- **obiettivi** da raggiungere o **preferenze** sugli stati del mondo;
- **abilità**, azioni primitivi di cui è capace.

Nella scatola nera si rappresenta un suo *stato interno delle credenze*, o **belief state**, comprendente la rappresentazione delle cose che si ritengono vere circa l'ambiente, cosa si è imparato, obiettivi intermedi presenti e futuri.

Il funzionamento di un KBS intelligente può essere schematizzato in due fasi, come nella seguente figura:

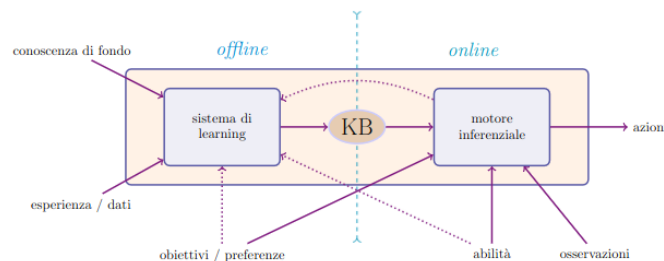


Figura 3: Funzionamento di un KBS

La KB viene coinvolta in momenti diversi:

- *Offline*, viene costruita/integrata usando conoscenza pregressa ed esperienze passate: spesso servono molti dati e conoscenza generale. Tale attività nei sistemi esperti non è automatizzata;
- *Online*, si prendono decisioni/compiono azioni e si aggiorna la KB; nel ragionamento si usano KB, osservazioni, obiettivi e abilità del sistema.

Una KB è utile nel futuro come *long-term memory*: viene appresa dai dati e dall'esperienza pregressa; un belief state, *short-term memory*, è un modello dell'ambiente attuale necessario tra successivi intervalli di tempo.

Uno schema di un KBS dettagliato è il seguente:

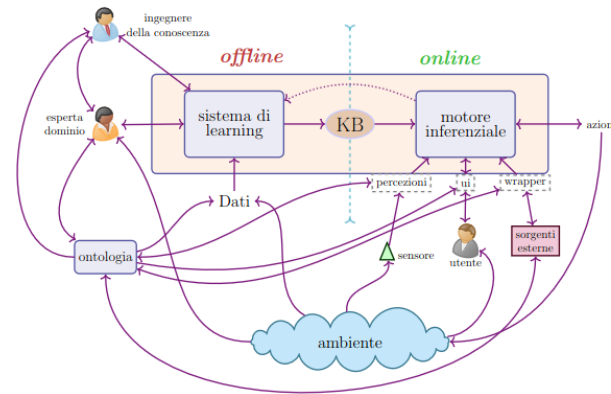


Figura 4: Schema dettagliato di un KBS

1.6. Domini applicativi

Tipici domini applicativi sono i seguenti:

- *robotica*;
- *diagnostica*: *assistente* che aiuta a capire i problemi di altri sistemi software e suggerisce rimedi;
- *tutoring*: *sistema* di aiuto agli studenti che interagisce fornendo informazioni su un argomento di interesse, assegnando compiti/test e valutando le capacità; deve comprendere la materia, lo studente e come questo impari;
- *e-commerce*: assistente agli acquisti di beni e servizi per conto dell'utente che sa riconoscere richieste e preferenze; sa trovare compromessi tra obiettivi distinti.

RICERCA DI SOLUZIONI IN SPAZI DI STATI

2.1. Risoluzione di problemi mediante ricerca

La risoluzione di diversi problemi spesso consiste nella *ricerca* in uno specifico modello del mondo. Considerando il caso di un sistema che, dato un *obiettivo* da raggiungere, ragiona su un *modello* del mondo fatto di *stati*, in *assenza di incertezza*.

Per raggiungere questo scopo, si adotta una rappresentazione *piatta*, ovvero relativa a un singolo livello in una gerarchia.

L'*astrazione* del problema prevede la ricerca di un *percorso* che vada da un nodo di partenza a uno dei nodi-obiettivo, detto *goal*, in un *grafo orientato*.

Esempio 3. (Navigatore) L'obiettivo è la ricerca del *miglior percorso* da un luogo a un altro: il più corto; quello di minimo costo; il più veloce; il più attrattivo, ecc. Ogni *stato* include informazioni su: localizzazione, direzione, velocità, mezzo di trasporto utilizzato e così via.

Una strategia di ricerca comune a molti problemi in AI è la seguente: il sistema computa su una *rappresentazione interna*. Il sistema riceve solo una descrizione di *cosa* rappresenti una soluzione; *come* ottenerla sarà il compito di un algoritmo di ricerca. Spesso questi problemi e i relativi algoritmi risolutivi sono NP-completi. Tali problemi risultano *difficili* anche per agenti umani. Spesso non esistono soluzioni ottimali per cui ci si accontenta di soluzioni *soddisfacenti* in mancanza d'una *struttura* mappabile su caratteristiche del mondo fisico.

2.2. Spazi di stati

Possiamo formulare la nozione di azione intelligente in termini di uno **spazio di stati**. Uno *stato* contiene tutte le informazioni necessarie per predire gli effetti di un'azione e per determinare se soddisfi l'obiettivo. Una ricerca su uno spazio di stati assume le seguenti nozioni:

- Gli agenti hanno una conoscenza perfetta dello spazio di stati.
- In qualsiasi momento, conosce in quale stato si trova; il mondo è quindi pienamente osservabile.
- L'agente ha un insieme di azioni con effetti deterministici conosciuti.
- L'agente ha un obiettivo da raggiungere e può determinare se uno stato soddisfa l'obiettivo.

Una **soluzione** di un problema di ricerca è una sequenza di azioni che porterà l'agente dallo stato corrente allo stato obiettivo.

Un **problema di ricerca su uno spazio di stati** comprende:

- un insieme di stati
- uno stato distinto detto *stato di partenza*
- per ogni stato, un insieme di azioni disponibili all'agente che si trova in tale stato
- una **funzione di azione** che, dato uno stato corrente e un'azione (s, a) , ritorna un nuovo stato in cui si transita per effetto dell'azione
- un **goal** che è una funzione booleana, $goal(s)$, che ritornerà *true* se s è uno stato che soddisfa l'obiettivo e, in tal caso, s è lo **stato obiettivo**

- un criterio che specifica la qualità di una soluzione accettabile; una **soluzione ottimale** massimizza tale criterio, ad esempio se ammette *qualsiasi* sequenza di azioni che porti a uno stato-obiettivo; ad esempio associando *costi* alle diverse azioni da cui discende un criterio del *minimo costo totale* delle azioni; a volte, sono soddisfacenti anche soluzioni sub-ottimali, ad esempio quelle con un costo aggiuntivo del +10% rispetto a quello di una soluzione ottimale.

Esempio 4. (Robot-consegne) Un robot che debba effettuare consegne in un edificio dovrà essere in grado di risolvere *problemi di ricerca* di percorsi da un luogo ad un altro in un dato piano.

Occorrerà definire:

- gli *stati*: posizioni distinte;
- le *azioni*: spostamenti da un luogo a un altro nelle vicinanze;
- il *problema* specifico: portarsi da $\square$ a $\square$ (unico stato-obiettivo);
- la *soluzione*: sequenza di spostamenti.

Possibili *estensioni* dei problemi di ricerca riguardano:

- la possibilità di sfruttare una *struttura interna* agli stati: ad esempio nei video-game;
- il caso di stati *non* completamente *osservabili*: ad esempio il robot-consegne non conosce la posizione iniziale delle consegne, oppure un sistema di tutoring può non conoscere bene le attitudini di uno studente;
- l'aggiunta di azioni *stocastiche*: ad esempio possibilità di commettere errori oppure mancato apprendimento da parte dello studente di un argomento;
- la possibilità di definire, invece degli stati finali, *preferenze aggiuntive* complesse, in termini di *ricompense* o *punizioni*.

Esempio 5. (Videogame) Si consideri lo spazio nella seguente figura.

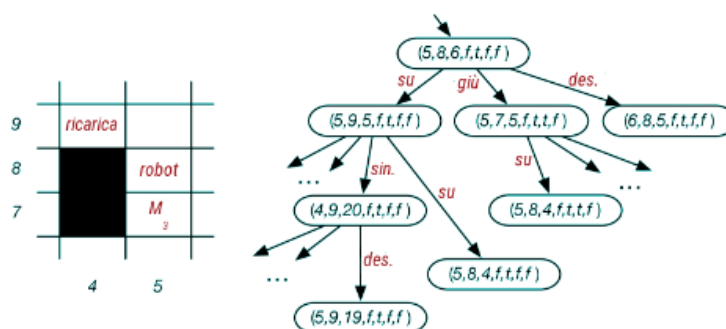


Figura 5: Spazio di stati del problema.

In tal caso definiamo:

- il *modello*: griglia nella quale ci si può muovere di una casella nelle 4 direzioni se non si è bloccati da un muro (caselle scure);
- l'*obiettivo*: raccogliere 4 monete $M_1, \dots, M_4$, ognuna in una posizione iniziale nota, ad esempio C_3 in $(5, 7)$;
- il *costo*: un'unità di energia per ogni passo: nessuna mossa è possibile senza energia; ricarica (+16 unità) presso una casella *ricarica*, ad esempio in $(4, 9)$;
- lo *stato*: dato da una posizione x, y , dalle unità di carburante u possedute e da 4 flag c_i che indicano il possesso della relativa moneta: $(x, y, u, m_1, m_2, m_3, m_4)$;
- possibili stato-obiettivo $(5, 7, ?, t, t, t, t)$ dove ? indica un quantitativo qualsiasi di carburante.

2.3. Ricerca su Grafo

Per risolvere il problema, si definisce lo spazio di ricerca e si applica un algoritmo di ricerca. Come detto precedentemente, molti compiti possono essere ricondotti alla *ricerca* di percorsi in un grafo. Un modello *astratto* della soluzione sarà *indipendente* dal particolare dominio applicativo di interesse. Dato un *grafo orientato* fatto di nodi connessi da archi, si dovrà trovare un *percorso* da un nodo di partenza a uno obiettivo. Ci possono essere più modi per rappresentare uno stesso problema.

Un **grafo orientato** *esplicito* consiste di:

- un insieme di *nodi* N , anche non finito;
- un insieme di *archi* A , ossia di coppie *ordinate* di nodi, anche non infinito;
 - l'arco n_1, n_2 si dirà **uscente** da n_1 ed **entrante** in n_2
 - si dirà che il nodo n_2 è un **vicino** del nodo n_1 se e solo se $\exists \langle n_1, n_2 \rangle \in A$
- un **percorso** dal nodo s al nodo g , denotato da $\langle n_0, n_1, \dots, n_k \rangle$, è una *sequenza* di nodi tale che se $s = n_0, g = n_l$ e $\forall i = 1, \dots, k : \langle n_{i-1}, n_i \rangle \in A$; in alternativa, si può indicare una *sequenza di archi* $\langle n_0, n_1 \rangle, \langle n_1, n_2 \rangle, \dots, \langle n_{k-1}, n_k \rangle$ o anche *sequenza di etichette* su tali archi: $\langle n_0, \dots, n_i \rangle$ *parte iniziale* di $\langle n_0, n_1, \dots, n_k \rangle$, con $i \leq k$.

Nel grafo esiste un insieme di *nodi-obiettivo* che si possono identificare anche tramite il predicato $goal(\cdot)$, funzione booleana definita sui nodi. Una **soluzione** è un percorso dal nodo di partenza a uno obiettivo.

In alcuni casi è associato un **costo** a ogni arco: dato $\langle n_i, n_j \rangle$

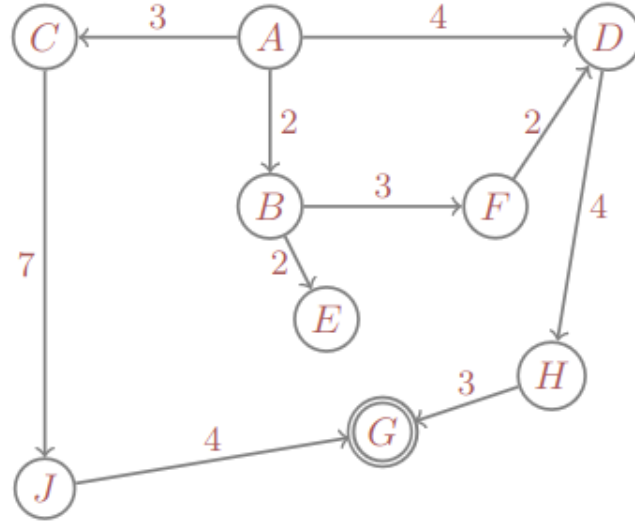
$$cost(\langle n_i, n_j \rangle) \in [0, +\infty[$$

estendibile anche ai percorsi: dato $p = \langle n_0, n_1, \dots, n_k \rangle$

$$cost(p) = \sum_{i=1}^k cost(\langle n_{i-1}, n_i \rangle)$$

Una soluzione **ottimale** p avrà costo minimo: $\nexists p'$ soluzione con $cost(p') < cost(p)$.

Esempio 6. (Robot consegne, continuo.) Ricerca di un percorso da A a G nel mondo rappresentato nella seguente figura.



- $N = \{A, B, C, D, E, F, G, H, J\}$
- $A = \{\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle, \langle B, E \rangle, \langle B, F \rangle, \langle C, J \rangle, \dots\}$ dove: E non ha vicini; C ha come vicino solo J ; A ha i vicini B, C e D ;
- vi sono 3 percorsi da A a G : $\langle A, D, H, G \rangle, \langle A, C, J, G \rangle, \langle A, B, F, D, H, G \rangle$; se A fosse nodo di partenza e G il nodo-obiettivo, ognuno di tali percorsi sarebbe una soluzione.

Un **ciclo** è un percorso non vuoto in cui primo e ultimo nodo coincidono: $\langle n_0, n_1, \dots, n_k \rangle$ tale che $k > 0$ e $n_0 = n_k$.

Un **grafo aciclico orientato** (*directed acyclic graph*, DAG) è un grafo orientato senza cicli. Un **albero** è un DAG con un solo nodo, la **radice**, senza archi entranti. I nodi senza archi uscenti sono le sue **foglie**.

2.4. Algoritmo di ricerca generico

Un algoritmo di ricerca generico è un algoritmo indipendente dalla strategia di ricerca dal grafo. Dato un grafo, si esplorano *incrementalmente* percorsi dai nodi di partenza verso nodi-obiettivo.

Si usa una struttura dati per rappresentare la **frontiera**, detta *fringe*, dei percorsi già esplorati. Inizialmente la frontiera contiene percorsi costituiti dai soli *nodi di partenza*. Successivamente si effettua l'*espansione* dei percorsi nella frontiera verso nodi inesplorati, fino a incontrare un nodo-obiettivo:

- si seleziona un percorso (rimuovendolo dalla frontiera);
- si estende il percorso con ogni arco uscente dall'ultimo nodo;
- si aggiungono alla frontiera i percorsi ottenuti.

Algoritmo 1: Algoritmo di ricerca generico

```

1  procedure Search( $G, s, goal$ )
2
3  Input
4       $G$ : grafo con insiemi di nodi  $N$  e di archi  $A$ 
5       $s$ : nodo di partenza
6       $goal$ : funzione booleana sui nodi
7  Output
8      percorso da  $s$  a un nodo per il quale  $goal$  sia vera
9      oppure  $\perp$  quando non ci sono percorsi / soluzioni
10 Local
11      $frontiera$ : insieme di percorsi
12
13      $frontiera \leftarrow \{\langle s \rangle\}$ 
14     while  $frontiera \neq \emptyset$  do
15         selezionare (e rimuovere)  $\langle n_0, \dots, n_k \rangle$  da  $frontiera$ 
16         if  $goal(n_k)$  then
17             return  $\langle n_0, \dots, n_k \rangle$ 
18          $frontiera \leftarrow frontiera \cup \{\langle n_0, \dots, n_k, n \rangle : \langle n_k, n \rangle \in A\}$ 
19     return  $\perp$ 

```

Come descritto nell'Algoritmo di ricerca generico, l'inizializzazione parte dal nodo s .

La selezione *non è deterministica*, ha solo impatto sull'efficienza quindi una data strategia di selezione determina il percorso da scegliere. Il $\perp$ annidato può essere interpretato come *temporaneo*, continuando si trovano strade alternative; $\perp$ indica che non vi sono soluzioni; la condizione $goal(n_k)$ viene testata dopo la selezione della frontiera e non all'aggiunta del nuovo nodo:

1. a volte esiste un *arco* verso un nodo-obiettivo ma di *costo elevato*: non sempre conviene terminare restituendo subito il percorso trovato in quanto potrebbe esserci un percorso di costo inferiore;
2. va tenuto conto del fatto che lo stesso test $goal()$ può essere *costoso*; un percorso che termini con un nodo che non sia un nodo-obiettivo e che non abbia vicini andrebbe rimosso.

2.5. Strategie di ricerca non informate

Il problema specifica il grafo e l'obiettivo mentre la **strategia di ricerca** determina il percorso da selezionare dalla frontiera. Le **strategie non informate** non prendono in considerazione la posizione dell'obiettivo. Se si considera un costo unitario per ogni arco, possibili strategie sono la ricerca *in ampiezza*, quella in *profondità* o l'*approfondimento iterativo*. Se è disponibile una funzione di costo, allora si usano strategie dei *costi minimi*.

2.5.1. Ricerca in ampiezza

Nella **ricerca in ampiezza**, detta *breadth-first search*, *BFS* la frontiera è implementata con una *coda*, ossia una struttura FIFO. Si seleziona il *primo* percorso aggiunto. I percorsi sono generati nell'ordine del numero di archi contenuti. A ogni passo, si seleziona uno dei percorsi più corti.

Esempio 7. Dal grafo in figura Figura 6, ignorando i costi, si ottiene il grafo nella seguente figura:

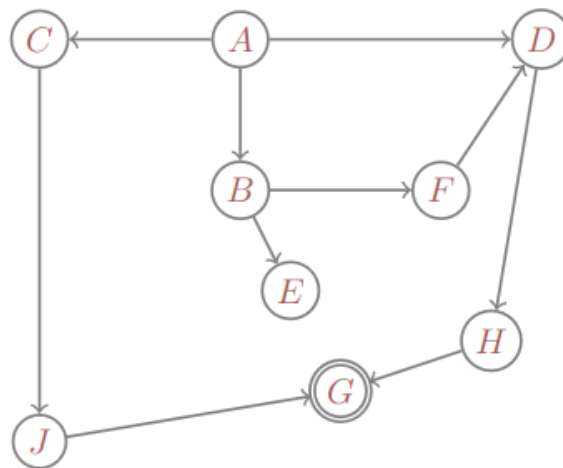


Figura 7: Grafo della figura Figura 6 ma senza informazioni sul costo

In questa figura A è il nodo di partenza e G è il nodo obiettivo. Con la BFS:

- frontiera iniziale: $[\langle A \rangle]$;
- estendendo $\langle A \rangle$ con i suoi vicini si ottiene:

$$[\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle]$$

nodi a un arco di distanza da A ;

- espandendo questi percorsi, nell'ordine si ha:

$$[\langle A, B, E \rangle, \langle A, B, F \rangle, \langle A, C, J \rangle, \langle A, D, H \rangle]$$

percorsi da A di lunghezza 2;

- dopo l'espansione dei percorsi del passo precedente la frontiera diventa:

$$[\langle A, B, F, D \rangle, \langle A, C, J, G \rangle, \langle A, D, H, G \rangle]$$

- selezionando, ad esempio, $\langle A, C, J, G \rangle$ questo percorso viene restituito come soluzione.

La BFS risulta utile quando:

- non si hanno problemi di spazio;
- si cerca una soluzione con numero di archi *minimale*.

La BFS è *poco utile* quando:

- tutte le soluzioni sono associate a percorsi lunghi;
- è disponibile conoscenza euristica;
- il grafo viene generato dinamicamente.

2.5.2. Ricerca in profondità

Nella **ricerca in profondità**, detta *depth-first search*, DFS, la frontiera è implementata con una *pila*, struttura LIFO: gli elementi vengono aggiunti uno alla volta e quello selezionato e prelevato sarà l'ultimo aggiunto.

Partendo dalla radice, i nodi sono considerati come ordinati da sinistra a destra: il vicino più a sinistra sarà aggiunto in cima per *ultimo*. L'ordine di espansione non dipende dalla posizione dei nodi-obiettivo.

Con una pila, la ricerca procede in profondità:

- *completamento* di un *singolo percorso* prima di provare alternative;
- questo comporta il **backtracking**: si seleziona una prima alternativa per ogni nodo, *tornando indietro* all'opzione successiva solo dopo aver tentato tutti i completamenti;
- l'*ordine di aggiunta* dei vicini alla frontiera/pila non è specificato: questo impatta sull'efficienza e può essere *statico*, ossia prefissato, ovvero *dinamico*, dipendendo dall'obiettivo;
- per una procedura DFS alternativa si veda la figura (inserire figura slide).

Esempio 8. (DFS sul grafo in Figura 7) La frontiera come lista con il primo percorso in *cima*. Si parte da $[\langle A \rangle]$. Le frontiere successive sono:

- $[\langle A, B \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, E \rangle, \langle A, B, F \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, F \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, F, D, H \rangle, \langle A, C \rangle, \langle A, D \rangle]$
- $[\langle A, B, F, D, H, G \rangle, \langle A, C \rangle, \langle A, D \rangle]$

Il percorso in cima viene restituito come *soluzione*.

La DFS è *appropriata* in caso di:

- limitazioni di spazio;
- presenza di molteplici soluzioni, anche se costituite da percorsi lunghi;
- ideale quando *tutti* portano a una soluzione;
- oppure se l'ordine di aggiunta dei vicini può essere variato in modo da trovare una soluzione al *primo tentativo*.

La DFS risulta *inefficiente* quando:

- sono possibili percorsi infiniti in caso di grafo infinito o contenente cicli;
- pur esistendo soluzioni alternative poco profonde, la ricerca si attarda su percorsi più lunghi.

2.5.3. Approfondimento iterativo

L'obiettivo dell'algoritmo **ITERATIVE DEEPENING** della DFS è combinare l'efficienza in spazio di DFS con l'ottimalità di BFS. L'idea è quella di non memorizzare ma ricalcolare gli elementi della frontiera di BFS usando DFS che usa meno spazio.

DFS effettua una ricerca limitata fino a una data *profondità* come per le iterazioni di BFS. In caso di fallimento si eliminano i percorsi via via calcolati ripartendo, se necessario, dopo aver aumentato il limite. Si parte cercando fino a profondità 1, poi si costruiscono i cammini fino a lunghezza 2, poi quelli di lunghezza 3, ecc.... Se esiste, una soluzione verrà trovata, esplorando percorsi in tale ordine: uno con il numero minimo di archi sarà individuato per primo.

Il *fallimento* della ricerca in profondità limitata può essere:

- *innaturale*: per raggiungimento del limite di profondità e in tal caso la ricerca riparte, dopo aver incrementato il limite;
- *naturale*: quando l'intero spazio di ricerca è esaurito non esiste soluzione a nessun livello di profondità.

```

1  procedure ID_search(G, s, goal)
2  Input
3    G: grafo con insiemi di nodi N e di archi A
4    s: nodo di partenza
5    goal: funzione booleana sugli stati
6  Output
7    cammino da s a un nodo per il quale goal sia vero
8    ovvero  $\perp$  altrimenti
9  Local
10   raggiunta_max_profondita: boolean
11   limite_profondita: integer
12
13  procedure Ricerca_a_profondita_limitata( $\langle n_0, \dots, n_k \rangle$ , b)
14  Input
15     $\langle n_0, \dots, n_k \rangle$ : percorso
16    b: integer  $\{b \geq 0\}$ 
17  Output
```

```

18  percorso fino a nodo-obiettivo di lunghezza  $k + b$ 
19
20  if  $b > 0$  then
21    for each  $\langle n_k, n \rangle \in A$  do
22      risultato  $\leftarrow$  Ricerca_a_profondita_limitata( $\langle n_0, \dots, n_k, n \rangle$ ,  $b - 1$ )
23      if risultato  $\neq \perp$  then
24        return risultato
25    else if goal( $n_k$ ) then
26      return  $\langle n_0, \dots, n_k \rangle$ 
27    else if  $n_k$  ha vicini then {fallimento innaturale}
28      raggiunta_max_profondita  $\leftarrow$  true
29    return  $\perp$ 
30
31  limite_profondita  $\leftarrow$  0
32  repeat
33    raggiunta_max_profondita  $\leftarrow$  false
34    risultato  $\leftarrow$  Ricerca_a_profondita_limitata( $\langle s \rangle$ , limite_profondita)
35    if risultato  $\neq \perp$  then
36      return risultato
37    limite_profondita  $\leftarrow$  limite_profondita + 1
38  until not raggiunta_max_profondita
39  return  $\perp$  {fallimento naturale}

```

La procedura `Ricerca_a_profondita_limitata` implementa una DFS a profondità limitata: trova percorsi di lunghezza $k + b$, dove k è la lunghezza del percorso e $b \geq 0$; viene chiamato a profondità crescenti. I percorsi vengono trovati nello stesso ordine della BFS: si controlla di aver trovato un obiettivo solo se $b = 0$.

Per assicurare di fallire quando anche la BFS fallirebbe, si tiene traccia dei casi in cui un limite maggiore potrebbe aiutare a trovare una soluzione. La ricerca *fallisce innaturalmente* se ha esaurito tutto lo spazio di ricerca e non sono stati tagliati percorsi per limite raggiunto; se `raggiunta_max_profondita`, falsa al momento della chiamata, diventa vera alla fine, il limite può essere incrementato per la successiva iterata.

2.5.4. Ricerca di soluzioni a costo minimo

In caso di archi con *costi* non unitari si cerca la soluzione di *minimo costo totale*. Gli algoritmi precedenti non garantiscono soluzioni di costo minimo: il costo non viene preso in considerazione. BFS minimizza solo il *numero* degli archi: potrebbe esistere una soluzione alternativa, come un percorso più lungo ma di costo inferiore. L'algoritmo **LOWEST COST-FIRST SEARCH** (LcFS), detto anche *least-cost search* o *uniform-cost search*, si comporta come BFS ma con la selezione dei percorsi di costo minimo. Viene implementato usando una *coda con priorità* come frontiera ordinata da *cost*.

Esempio 9. Si consideri il grafo in figura Figura 6. Si espanderà sempre il percorso più a sinistra nella frontiera (costi dei percorsi indicati come pedici):

- $[\langle A \rangle_0]$ frontiera iniziale
- $[\langle A, B \rangle_2, \langle A, C \rangle_3, \langle A, D \rangle_4]$
- $[\langle A, C \rangle_3, \langle A, B, E \rangle_4, \langle A, D \rangle_4, \langle A, B, F \rangle_5]$
- $[\langle A, B, E \rangle_4, \langle A, D \rangle_4, \langle A, B, F \rangle_5, \langle A, C, J \rangle_{10}]$
- $[\langle A, D \rangle_4, \langle A, B, F \rangle_5, \langle A, C, J \rangle_{10}]$
- $[\langle A, B, F \rangle_5, \langle A, D, H \rangle_8, \langle A, C, J \rangle_{10}]$
- $[\langle A, B, F, D \rangle_7, \langle A, D, H \rangle_8, \langle A, C, J \rangle_{10}]$
- $[\langle A, D, H \rangle_8, \langle A, C, J \rangle_{10}, \langle A, B, F, D, H \rangle_{11}]$
- $[\langle A, C, J \rangle_{10}, \langle A, D, H, G \rangle_{11}, \langle A, B, F, D, H \rangle_{11}]$.

Dopo l'espansione di $\langle A, C, J \rangle$, si selezionerà $\langle A, D, H, G \rangle$, percorso di costo minimale da A a G .

Il fattore di ramificazione è *finito* se i *costi* sono *limitati* inferiormente da una costante positiva: ciò garantisce la soluzione ottimale, quando esiste. Il primo percorso trovato termina in un nodo-obiettivo ed è ottimale perchè si procede in ordine di costo.

In mancanza di un limite inferiore allora sono possibili percorsi *infiniti*, ad esempio con nodi $n_0, n_1, n_2, \dots$ con $\forall i > 0 : \langle n_{i-1}, n_i \rangle$ di costo $\frac{1}{2^i}$ esistono infiniti percorsi $\langle n_0, n_1, \dots, n_k \rangle$, con costo < 1 ; se $\exists \langle n_0, g \rangle, goal(g)$ con costo ≥ 1 , l'arco non verrebbe mai selezionato.

La complessità è esponenziale in spazio e tempo: genera tutti i percorsi con costo inferiore a quello della soluzione.

2.6. Ricerca Informata (Euristica)

2.6.1. Euristica

Tale tipo di ricerca prende in considerazione *informazioni sull'obiettivo*, nella selezione dei nodi da esplorare, attraverso l'uso di una **funzione euristica** h che associa a ogni nodo n un numero reale *non-negativo*, una stima del costo minimale di un percorso da n fino a un nodo-obiettivo.

L'euristica h si dice **ammissibile** se *sottostima* il costo reale: $h(n)$ minore o uguale rispetto al costo minimale effettivo del percorso da ciascun nodo a un nodo-obiettivo. Tale informazione approssimata è spesso immediatamente disponibile trovando un *compromesso* tra efficienza del suo calcolo e accuratezza della stima. Tipicamente, risolvendo una forma semplificata del problema, si possono poi usare i costi effettivi del problema semplificato come euristica nella soluzione del problema originario.

Esempio 10. (Problema nel grafo in Figura 6) Considerando come *euristica* la distanza in linea retta tra nodo e obiettivo più vicino, quindi supponendo che tali distanze siano ad esempio:

	A	B	C	D	E	F	G	H	J
$h(\cdot)$	7	5	9	6	3	5	0	3	4

h risulta ammissibile e in particolare: essa è

- *esatta* per il nodo H ;
- *molto sottostimata* per il nodo B : sembra vicino al nodo-obiettivo (costo 5), ma raggiungere l'obiettivo costa molto di più;
- *ingannevole* per il nodo E : sembra vicino al nodo-obiettivo (costo 3) ma non permette di raggiungere l'obiettivo.

2.6.2. DFS Euristica e Greedy best-First Search

L'euristica h può essere *estesa* al caso dei percorsi:

$$h(\langle n_0, \dots, n_k \rangle) = h(n_k)$$

La **DFS Euristica** usa h per ordinare i *vicini* aggiunti alla pila/frontiera della DFS standard. I vicini vengono aggiunti in modo che il *migliore* vada in cima. Tale scelta è comunque *locale*: si esplorano i percorsi che estendono quello selezionato prima di tentarne altri. Si presentano problemi analoghi al caso della DFS: non c'è garanzia di terminazione e ritrovamento di una soluzione ottimale.

Si potrebbe considerare la *best-first search* che sfrutta un'euristica h da minimizzare nella scelta del prossimo cammino da espandere. In particolare, nella *greedy best-first search*, GbFS, si sceglie il percorso della frontiera con valore di h minimo. Purtroppo potrebbe seguire cammini apparentemente promettenti che, però, potrebbero continuare a estendersi indefinitamente.

2.6.3. Ricerca A^*

L'algoritmo A^* combina idee da LcFS e GbFS. Nella selezione del percorso da espandere si considerano: il costo effettivo del percorso parziale che dal nodo iniziale arriva al nodo corrente e l'euristica, ossia una stima del percorso dal nodo corrente fino a un obiettivo.

Per ogni percorso $p = \langle s, \dots, n \rangle$ della frontiera, si stima il costo di un percorso che passi per p e progesua fino a un nodo-obiettivo g :

$$f(p) = cost(p) + h(p)$$

dove $cost(p)$ rappresenta il costo effettivo di p e $h(p)$ è una stima del costo del cammino da n a g .

Si segue lo schema dell'algoritmo di ricerca generico in Algoritmo di ricerca generico e si rappresenta la *frontiera* con una coda con priorità, ordinata da $f(p)$. Se h è ammissibile allora $f(p)$ non sovrastima il cost d'un percorso completo che includa p . A^* può seguire molti perocrsi ma alla fine ne trova uno di costo minimo. Altri

possono sembrare (provvisoriamente) di costo inferiore. A^* migliora le prestazioni degli algoritmi da cui origina.

Esempio 11. (A^* su grafo ed euristica) Si consideri il grafo in figura:

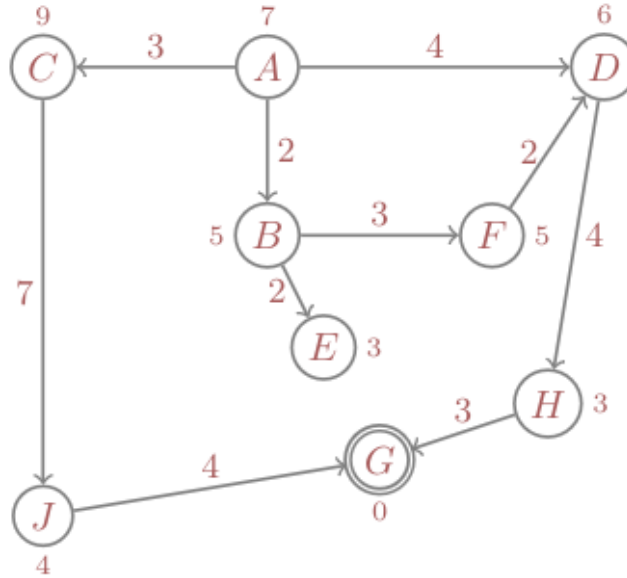


Figura 8: Problema con costi effettivi sugli archi e valori dell'euristica sui nodi

Sequenza delle frontiere. Il pedice indica il valore di $f(p)$.

- $[\langle A \rangle_7]$ iniziale, essendo $h(A) = 7$ e $cost(\langle \rangle) = 0$
- $[\langle A, B \rangle_7, \langle A, D \rangle_{10}, \langle A, C \rangle_{12}]$
- $[\langle A, B, E \rangle_7, \langle A, B, F \rangle_{10}, \langle A, D \rangle_{10}, \langle A, C \rangle_{12}]$
- $[\langle A, B, F \rangle_{10}, \langle A, D \rangle_{10}, \langle A, C \rangle_{12}]$
- $[\langle A, D \rangle_{10}, \langle A, C \rangle_{12}, \langle A, B, F, D \rangle_{13}]$
- $[\langle A, D, H \rangle_{11}, \langle A, C \rangle_{12}, \langle A, B, F, D \rangle_{13}]$
- $[\langle A, D, H, G \rangle_{11}, \langle A, C \rangle_{12}, \langle A, B, F, D \rangle_{13}]$

La soluzione è $\langle A, D, H, G \rangle$ di costo 11.

Un algoritmo è **ammissibile** se, quando esistono soluzioni, ne trova sempre una ottimale.

Proposizione 1. A^* è ammissibile se:

- il fattore di ramificazione è limitato;
- i costi degli archi sono maggiori di un certo $\epsilon > 0$
- h è ammissibile, non supera il costo minimale di percorsi da ciascun nodo a un obiettivo.

A^* garantisce che la prima soluzione sarà ottimale, anche in presenza di cicli, ma non assicura che ciascun nodo intermedio selezionato dalla frontiera sia un cammino ottimo.

Importanza dell'euristica

Il miglioramento dell'efficienza in A^* è dato dall'uso di h . Sia c il costo del percorso di costo minimo. Se h è ammissibile, A^* espande ogni percorso dal nodo di partenza nell'insieme

$$\{p \mid \text{cost}(p) + h(p) < c\}$$

più alcuni percorsi nell'insieme

$$\{p \mid \text{cost}(p) + h(p) = c\}$$

Per l'efficienza di A^* , h è tanto migliore quanto più riduce la cardinalità del primo insieme.

2.6.4. Algoritmo Branch and Bound

L'idea di questo algoritmo è combinare l'efficienza in spazio delle strategie in profondità con informazione euristica. Risulta efficace quando esistono molti cammini verso un nodo-obiettivo. Questo algoritmo assume che $h(n)$ sia ammissibile.

Questo algoritmo memorizza il percorso di costo minimo verso un nodo-obiettivo trovato e il suo costo che diventa un *limite*: un percorso $p : \text{cost}(p) + h(p) \geq \text{limite}$ aggiornando *limite*; si continua poi a cercare un'eventuale soluzione migliore.

Si genera così una sequenza di soluzioni via via migliori restituendo quella finale: l'algoritmo viene tipicamente implementato usando la DFS per la ricerca in profondità limitata dal valore di *limite*.

```

1  procedure DFBranchAndBound(G, s, goal, h, limite_0)
2  Input
3    G: grafo con nodi N e archi A
4    s: nodo di partenza
5    goal: funzione di test dei nodi
6    h: euristica sui nodi
7    limite_0: limite iniziale ( $\infty$  se non specificato)
8  Output
9    percorso di costo minimo da s a un nodo obiettivo se esiste una
10   soluzione di costo minore di limite_0, oppure  $\perp$ 
11  Local
12   percorso_migliore: percorso o  $\perp$ 
13   limite: numero reale non negativo
14
15   procedure cbsearch( $\langle n_0, \dots, n_k \rangle$ )
16     if  $\text{cost}(\langle n_0, \dots, n_k \rangle) + h(n_k) < \text{limite}$  then
17       if goal( $n_k$ ) then
18         percorso_migliore  $\leftarrow \langle n_0, \dots, n_k \rangle$ 
```

```

19     limite ← cost(⟨n_0,...,n_k⟩)
20   else
21     for each ⟨n_k,n⟩ ∈ A do
22       cbsearch(⟨n_0,...,n_k,n⟩)
23
24   {main}
25   percorso_migliore ← ⊥
26   limite ← limite_0
27   cbsearch(⟨s⟩)
28   return percorso_migliore

```

La procedura *cbsearch*, *cost-bounded search*, comunica attraverso variabili globali; inizialmente, *limite* è impostata a limite_0 , stima per eccesso del costo d'una soluzione ottimale.

DFBranchAndBound restituirà, se esiste, una soluzione ottimale con costo inferiore a limite_0 . Se limite_0 supera di poco il costo minimo, l'algoritmo non espanderà più archi di A^* . La variabile limite_0 serve a far eliminare percorsi di costo maggiore: trovato un percorso completo, si esplorano solo percorsi con valore di f inferiore a quello del percorso trovato, ossia esattamente i percorsi esplorati da A^* quando trova una soluzione. Quando restituisce $\perp$, se $\text{limite}_0 = \infty$ allora non ci sono soluzioni, se invece limite_0 è finito allora non ci sono soluzioni di costo inferiore a limite_0 .

Combinato con ID l'algoritmo può incrementare via via il limite fino a trovare una soluzione ovvero può mostrare che non c'è soluzione.

Esempio 12. (Robot-consegne) In un problema più complesso, si considerino:

- *stati* comprendenti le consegne da fare;
- il *costo* dato dalla distanza totale per tutte le consegne;
- un'*euristica* possibile pari al massimo tra:
 1. la distanza massima per consegne ancora da fare non caricate: per ognuna, la distanza dalla sua posizione e, da qui, fino alla destinazione;
 2. la distanza della destinazione più lontana per le consegne trasportate.

Tale massimo non sovrastima il costo: è la soluzione per un problema semplificato che non considera muri e tutte le altre consegne da fare tranne la più difficile; è appropriato: vanno consegnate quelle trasportate, passate a prendere quelle ancora non caricate e portate a destinazione.

Se il robot può trasportare una sola consegna, un'*euristica* dovrebbe sommare le distanze per il trasporto di ognuna più la distanza dalla più vicina: non è detto che sarà effettuata per prima.

Esempio 13. (Navigatore) Per la minimizzazione del tempo:

- possibile euristica: distanza in linea retta dalla locazione corrente al nodo-obiettivo divisa per la velocità massima, assumendo che ci si possa dirigere a destinazione alla massima velocità;
- euristica più sofisticata: date velocità massime diverse per autostrade e strade locali, si sceglie il massimo tra:
 1. minimo tempo stimato per andare a destinazione su strade locali lente;
 2. minimo tempo utile a raggiungere un'autostrada da una strada locale, per poi raggiungere un luogo vicino alla destinazione e arrivarci da strada locale.

2.6.5. Progettazione della funzione Euristica

Un'euristica ammissibile è una funzione non negativa h definita sui nodi, tale che dato un nodo n , $h(n)$ non è mai superiore al costo reale del percorso ottimale da n a un nodo-obiettivo.

Una *procedura standard* per definire un'euristica può essere descritta come segue:

- Si trovano soluzioni di un problema più semplice, ossia con *meno vincoli*, quindi spesso molto più facile da risolvere; una sua soluzione ottimale avrà un costo pari o inferiore a una soluzione del problema originario.
- Si considerino problemi di ricerca su mappe semplificate rappresentati da grafi, in cui si sia *vincolati* a muoversi lungo i dati archi: supponendo che il loro costo sia proporzionale alla lunghezza dei collegamenti stradali, un'euristica ammissibile è costituita dalla *distanza euclidea* tra punti/nodi.

Inserire esempi slide

Un problema in forma semplificata può essere risolto anche attraverso una ricerca più semplice rispetto al problema originario: ricerca ripetuta anche più volte e per tutti i nodi.

Conviene memorizzare (in una memoria cache) tali risultati in un **DB di pattern** che associ ai nodi del problema semplificato il valore dell'euristica.

Nel problema semplificato, spesso si hanno meno nodi: più nodi originari sono mappati su uno stesso nodo di quello semplificato come fosse una *classe d'equivalenza*.

2.7. Potatura dello Spazio di Ricerca

Si possono migliorare gli algoritmi di ricerca considerando i diversi percorsi che possono attraversare un nodo. Non tutti sono utili a risolvere il problema per cui gli altri possono essere subito eliminati.

Essenzialmente vi sono due tipi di **pruning**, ovvero *potatura*, con relative strategie:

- *potare i cicli* per trovare percorsi dal costo minimo;
- *potare cammini molteplici*: per ogni nodo si considererà un solo percorso che lo attraversi, eliminando tutti gli altri.

2.7.1. Potatura dei Cicli

Alcuni dei metodi di ricerca visti in precedenza potrebbero rimanere intrappolati in cicli senza possibilità di uscita anche nel caso di grafi finiti.

Per avere la garanzia di trovare una soluzione in caso di *grafo finito* non andranno presi in considerazione vicini già presenti nel percorso corrente.

Nella **potatura dei cicli**, detta *cycle / loop pruning*, *CP*, si prevede un controllo addizionale preventivo: dato un nodo da aggiungere, va testata l'occorrenza nel percorso.

Nello schema di **Algoritmo di ricerca generico** si aggiungono alla frontiera solo percorsi $\langle n_0, \dots, n_k, n \rangle$, tali che $n \notin \{n_0, \dots, n_k\}$, scartando il percorso selezionato; in alternativa si può effettuare il controllo dopo la selezione del nodo.

La complessità dei controlli aggiuntivi per la verifica della presenza di un nodo nel percorso corrente dipende dal metodo di ricerca adottato:

- *costante* per metodi (DF) che memorizzano un *unico percorso*; si può usare una funzione *hash* oppure si può associare un *bit* a ogni nodo, che sarà acceso quando viene aggiunto a un percorso e viene spento in caso di backtracking; per cui basterà non espandere nodi con il bit acceso; tale metodo funziona perchè si lavora su un solo percorso;
- *lineare* nella lunghezza del percorso corrente per metodi che gestiscono più percorsi (esponenziali in spazio): serve una ricerca per evitare di aggiungere al percorso parziale un nodo già presente.

2.7.2. Potatura di Percorsi Multipli

Spesso più di un percorso porta a uno stesso nodo. Occorre quindi *eliminare* dalla frontiera ogni percorso che porti a un nodo per il quale ne esista già un altro (di costo inferiore).

Una strategia di **multiple-path pruning** (MPP) può essere implementata gestendo una lista di nodi terminali di percorsi già esplorati, detta **closed list** o *explored set*: inizialmente essa è vuota poi, selezionato un percorso $\langle n_0, \dots, n_k \rangle$ della frontiera, se n_k è già nella lista esso può essere scartato, altrimenti si aggiunge n_k alla lista e si prosegue.

```
1 procedure SearchMPP(G, s, goal)
2   Input
3   - G: grafo con nodi N e archi A
4   - s: nodo di partenza
5   - goal: funzione booleana sui nodi
6   Output
7   - cammino da s a un nodo per cui goal è vero
8   - oppure  $\perp$  se non ci sono cammini soluzione
9   Variabili locali
10  - frontier: insieme di cammini
11  - explored: insieme di nodi esplorati
```

```

12
13 frontier := {s}
14 explored := {}
15 while frontier != {} do
16   seleziona e rimuovi  $\langle n_0, \dots, n_k \rangle$  da frontier
17   if  $n_k \notin \text{explored}$  then
18     explored := explored  $\cup$  { $n_k$ }
19     if goal( $n_k$ ) then
20       return  $\langle n_0, \dots, n_k \rangle$ 
21     frontier := frontier  $\cup$  { $\langle n_0, \dots, n_k, n \rangle : \langle n_k, n \rangle \in A$ }
22   return  $\perp$ 

```

Si noti che questa strategia non garantisce che non elimino un percorso di costo minimo. Per garantire di preservare le soluzioni ottimali vi sono alcune alternative:

- assicurare che il primo percorso trovato per un dato nodo sia ottimale quindi gli altri si possono eliminare;
- se nella frontiera si dovesse inserire un percorso $p = \langle s, \dots, n, \dots, m \rangle$, ma fosse già presente $p' = \langle s, \dots, n \rangle$ meno costoso della parte fino a n in p , si può eliminare p oppure sostituire in p tale parte con p' .

In LcFS, il primo percorso verso un dato nodo è quello di costo minimo. Pertanto potare altri percorsi non elimina un cammino a costo minimo per il nodo. Ciò assicura di poter trovare una soluzione ottimale.

A^* non garantisce che quando si seleziona un percorso verso un nodo per la prima volta, questo sia quello di costo minimo: il teorema sull'ammissibilità lo garantisce solo per i percorsi verso nodi-obiettivo; per quelli verso altri nodi dipende dalle proprietà dell'euristica.

Un'euristica consistente h (non negativa) soddisfa il vincolo:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

per ogni coppia di nodi n e n' .

Se $h(g) = 0$ per ogni nodo-obiettivo g , allora h sarà consistente e non sovrastimerà mai il costo dei percorsi da un nodo verso un obiettivo.

La consistenza è garantita se h soddisfa la **restrizione di monotonicità**:

$$h(n) \leq \text{cost}(n, n') + h(n')$$

per ogni arco $\langle n, n' \rangle$. Ciò è più facile da testare: dipende solo dagli archi, non dalle coppie di nodi considerate.

strategia	selezione dalla frontiera	garanzia soluzione	complessità (spazio)
BFS	primo nodo aggiunto	con meno archi	$O(b^d)$
DFS	ultimo nodo aggiunto	no	$O(bd)$
ID	n/a	con meno archi	$O(bd)$
G_{BFS}	$h(p)$ minimale	no	$O(b^d)$
LcFS	$cost(p)$ minimale	costo minimo	$O(b^d)$
A^*	$cost(p) + h(p)$ minimale	costo minimo	$O(b^d)$
DF B&B	n/a	costo minimo	$O(bd)$
IDA*	n/a	costo minimo	$O(bd)$

Tabella 1: Consuntivo dei diversi algoritmi: si considera la complessità in spazio, con d profondità e b limite superiore per il *fattore di ramificazione*.

Proposizione 2. *Data un'euristica consistente, MPP non impedisce ad A^* di trovare una soluzione ottimale.*

Quindi, nelle condizioni della proposizione sull'ammissibilità di A^* che garantiscono solo il ritrovamento di una soluzione ottimale, se l'euristica è consistente, anche A^* con MPP la potrà trovare.

Nella pratica, A^* include MPP per default.

Confronto fra MPP e CP

- MPP è più generale di CP: un ciclo può essere considerato come un altro percorso verso un dato nodo, destinato a essere potato;
- MPP può essere implementato in modo da poter richiedere un tempo costante;
- MPP è preferibile con metodi in ampiezza: virtualmente tutti i nodi considerati vanno memorizzati;
- CP è preferibile con le strategie in profondità, ad esempio DFS con Branch-and-Bound; con MPP+DFS c'è il rischio di crescita esponenziale per la lista chiusa.

2.7.3. Consuntiva sulle Strategie di Ricerca

Un consuntivo schematico sulle diverse strategie di ricerca e la relativa complessità in spazio è disponibile nella (linkare la tabella inserita prima).

Un'algoritmo di ricerca **completo** garantisce il ritrovamento di una soluzione quando esiste. Le strategie che trovano percorsi con numero archi o costo minimale sono complete: nel caso pessimo serve un tempo esponenziale nel numero di archi dei percorsi esplorati. L'esistenza di algoritmi completi con minore complessità dipende dalla risposta alla domanda

$$P \neq NP?$$

2.8. Strategia di Ricerca più sofisticate

Sono possibili alcune estensioni delle strategie di ricerca presentate:

- dato che decidere la direzione della ricerca impatta sull'efficienza, si può pensare a una ricerca retrograda per soluzioni ottimali da qualunque nodo;
- per determinare una buona euristica si può usare la *programmazione dinamica*: metodi basati sulla *decomposizione* del problema in un numero di problemi equivalenti ma di minore complessità.

2.8.1. Direzione della ricerca

Nella **ricerca in avanti**, *forward search*, si comincia da un nodo di partenza e si prosegue fino a raggiungere i nodi-obiettivo. Nella **ricerca retrograda**, *backward search*, si parte da un nodo-obiettivo e si usa il *grafo inverso* alla ricerca del non di partenza; i vicini del nodo-obiettivo considerati per inizializzare la frontiera sono tutti gli altri nodi-obiettivo: $\{n : goal(n)\}$.

La direzione di ricerca è indifferente quando:

1. il numero di nodi-obiettivi è finito;
2. per ogni nodo n si possono generare i *vicini* del **grafo inverso** $\{n' : \langle n', n \rangle \in A\}$.

Le dimensioni dello spazio di ricerca sono date da b^d , dove b è il fattore di ramificazione e d la lunghezza del cammino. Riducendo questi parametri si abbassa la complessità aumentando l'*efficienza*. È possibile che vi sia una differenza tra i fattori da ramificazione uscente ed entrante; in tal caso un principio generale per la scelta della direzione della ricerca è quello di considerare la direzione per cui b risulta inferiore.

2.8.2. Ricerca bidirezionale

Nella **ricerca bidirezionale** si riduce il tempo di ricerca procedendo in entrambe le direzioni: quando le due frontiere si intersecano, si deve ricostruire un singolo cammino dal nodo di partenza al nodo-obiettivo.

Come si è sicuro che le due frontiere s'incontrano? Nel caso della DFS è difficile date le ridotte dimensioni delle frontiere mentre nel caso della BFS è garantito.

Combinando DFS e BFS in direzioni opposte, si garantirebbe l'intersezione, ma la scelta per la direzione del cammino dipende. Per la BFS il fattore di tale scelta è il costo del mantenimento della frontiera (tempo proporzionale a b^k) mentre per la DFS è il costo della ricerca nella frontiera per avere un'intersezione; se si tratta di una ricerca simmetrica bidirezionale si è nell'ordine di $2b^{\frac{k}{2}}$, con risparmi in tempo, ma la complessità rimane esponenziale.

2.8.3. Ricerca in una gerarchia di astrazioni

L'idea è quella di astrarre la definizione del problema, rimuovendo dettagli: una soluzione parziale del problema potrà essere ottenuta da una soluzione per la

versione astratta. Con l'astrazione si punta a risolvere un problema in generale, lasciando da risolvere problemi più semplici e specifici.

Diverse sono le modalità di astrazione di problemi. Tutte le strategie di ricerca presentate sono utilizzabili ma le specifiche decomposizioni utili non risultano facili da individuare.

Esempio 14. (Ricerca a isole) Si consideri una ricerca fra i diversi piani di un edificio. Trovata la soluzione a livello di isola si passa a risolvere ricorsivamente i sotto-problemi in modo analogo:

- informazioni sulle soluzioni trovate a livelli più bassi possono servire a quelle per livelli più alti;
- ai livelli più alti si può usare tale informazione per riformulare un piano di ricerca;
- il processo tipicamente non garantisce soluzioni ottimali perchè considera solo alcune delle decomposizioni possibili.

2.8.4. Uso della programmazione dinamica

La **programmazione dinamica, PD**, fornisce un metodo generale di ottimizzazione basato sulla memorizzazione di soluzioni parziali: una soluzione per un problema più semplice già prodotta dev'essere solo ritrovata e non ricalcolata.

Si può ricorrere alla PD per la costruzione offline di un'**euristica perfetta**, *cost_to_goal()*, che fornisca il costo esatto di un cammino completo di costo minimo su frati finiti:

$$\text{cost_to_goal}(n) = \begin{cases} 0 & \text{se } \text{goal}(n) \\ \min_{\langle n, m \rangle \in A} [\text{cost}(\langle n, m \rangle) + \text{cost_to_goal}(m)] & \text{altrimenti} \end{cases}$$

calcolato usando LcFS + MPP sul grafo inverso a partire dai nodi-obiettivo: si cerca il cammino di costo minimo da ogni nodo verso i nodi-obiettivo e si conserva il valore di *cost_to_goal* per ogni nodo.

Una **policy** è una politica da seguire nella specifica dell'arco da considerare per ogni nodo; essa è **ottimale** se i suoi costi non sono mai superiori ai costi calcolati usando altre policy.

- *cost_to_goal* viene calcolata offline e usata nella costruzione di una policy: da *n* si dovrebbe andare al vicino *m* che minimizza $\text{cost}(\langle n, m \rangle) + \text{cost_to_goal}(m)$. Questa policy porta sempre a un obiettivo con un cammino di costo minimo partendo da qualsiasi nodo;
- sono presenti delle alternative:
 1. memorizzare il vicino per tutti i nodi offline, sfruttando le associazioni tra nodi nelle decisioni online sulle azioni, oppure
 2. fornire *cost_to_goal* pre-calcolata e calcolare i vicini online.

Riguardo la complessità si può dire in breve che:

1. PD è lineare in tempo e spazio rispetto alle dimensioni del grafo nella costruzione di *cost_to_goal*, tipicamente esponenziali nella lunghezza del percorso;
2. Data *cost_to_goal*, determinare l'arco migliore richiede tempo costante rispetto alla grandezza del grafo, essendoci un numero limitato di vicini per nodo.

La PD può servire a costruire euristiche per A^* e Branch-and-Bound: semplificando il problema fino a ottenere spazi di ricerca ridotti e trovando in tali spazi soluzioni di lunghezza ottimale; ciò definisce un DB di pattern usato nell'euristica per il problema originale.

La PD è utile quando i nodi-obiettivo possono essere elencati esplicitamente, la soluzione cercata è un cammino di costo minimo, il grafo ridotto è finito, con memoria sufficiente per contenere la tabella, l'obiettivo non cambia. La policy può essere riusata per i diversi nodi-obiettivo: si ammortizza il costo per produrre la tabella su diverse istanze dello stesso problema; tuttavia essa va calcolata per ogni diverso nodo-obiettivo.

RAGIONAMENTO CON VINCOLI

3.1. Variabili e vincoli

Vedremo come passare da problemi definiti su spazi di stati a problemi su spazi definiti da caratteristiche. Le *caratteristiche*, o *feature*, vengono descritte attraverso l'uso di **variabili**, spesso non indipendenti fra loro, e di **vincoli rigidi** che specificano combinazioni lecite di assegnazioni alle variabili, e/o **vincoli flessibili**, ossia funzioni che codificano le preferenze fra le diverse assegnazioni.

Il *ragionamento* si svolge generando assegnazioni che soddisfino i vincoli rigidi e ottimizzino i vincoli flessibili.

3.1.1. Variabili e Assegnazioni

Si considereranno problemi descritti in termini di variabili *algebriche* ossia simboli usati per denotare caratteristiche del mondo (reale o immaginario).

La notazione adotta nomi che iniziano per maiuscola: ogni variabile ha un **dominio** associato, denotato con $dom(X)$. Si considereranno *variabili discrete* con dominio finito o almeno enumerabile. Un altro tipo è quello delle *variabili continue*, ad esempio variabili con dominio $\mathbb{R}$.

Un'**assegnazione** è una funzione da un insieme di variabili ai loro domini: dato $\{X_1, X_2, \dots, X_k\}$ a X_i si assegna $v_i \in dom(X_i)$ per ogni $i = 1, \dots, k$:

$$X_1 = v_1, X_2 = v_2, \dots, X_k = v_k$$

Essendo una funzione a ogni variabile viene assegnato un solo valore. Un'**assegnazione totale** riguarda tutte le variabili, altrimenti si dice **parziale**. Un'assegnazione totale rappresenta uno stato del mondo, detto *mondo possibile*. Date n variabili con domini di cardinalità d , si hanno d^n assegnazioni totali. L'uso delle variabili offre un *vantaggio*: con poche variabili si possono descrivere molti stati:

- con 10 variabili binarie: $2^{10} \approx 10^3$ stati;
- con 20 variabili binarie: $2^{20} \approx 10^6$ stati;
- con 30 variabili binarie: $2^{30} \approx 10^9$ stati;
- con 100 variabili binarie: $2^{100} \approx 10^{30}$ stati.

Ragionare con 30 variabili è più facile che con un miliardo di stati ma anche con 100 variabili non ci sarebbero grossi problemi, mentre è impraticabile ragionare esplicitamente con 2^{100} stati.

Tuttavia molti problemi reali possono essere definiti solo in termini di migliaia o anche milioni di variabili, come le previsioni del meteo.

3.1.2. Vincoli

Dato un problema, le assegnazioni possono essere *ammissibili* o *non ammissibili*. Un **vincolo rigido** (*hard constraint*) specifica le assegnazioni lecite per una o più variabili e comprende:

- un **ambito**, detto *scope*, ossia l'insieme S di variabili coinvolte con una sua arietà $|S|$
- la **condizione**, una funzione booleana sulle assegnazioni alle variabili del vincolo che dovrà risultare vera solo per assegnazioni lecite.

Seguono alcuni esempi di vincoli di diverse arietà:

- $B \leq 3$ *unario*;
- $A \leq B$ *binario*;
- $A + B = C$ *ternario*.

La *definizione* di un vincolo può essere: **intensionale**, ossia in termini di formule logiche; **estensionale**, come elencazione delle assegnazioni lecite, come con le relazioni ovvero le tabelle di tuple nei DB relazionali.

Dati il vincolo c con ambito S e l'assegnazione A su variabili contenute in S , anche non tutte, si dirà che A soddisfa c se la condizione è vera per A ristretta all'ambito di c ovvero che A **viola** c in caso contrario.

Esempio 15. (Gita) In una gita di quattro giorni, tre *attività* da svolgere possono essere rappresentate da variabili A, B, C tutte con identico dominio $\{1, 2, 3, 4\}$.

Si consideri il seguente vincolo in forma intensionale su $\{A, B, C\}$:

$$(A \leq B) \wedge (B < 3) \wedge (B < C) \wedge \neg(A = B \wedge C \leq 3)$$

esso richiede che A non possa seguire B , B sia svolta prima del 3° giorno e comunque prima di C e, infine, che se A è concomitante con B , C si svolga dopo il 3° giorno. Lo stesso vincolo, definito estensionalmente elenca le assegnazioni lecite:

A	B	C
2	2	4
1	1	4
1	2	3
1	2	4

Ad esempio è soddisfatto da $\{A = 1, B = 2, C = 3, D = 3, E = 1\}$ in quanto in tabella si trova $\{A = 1, B = 2, C = 3\}$, ristretta al suo ambito.

3.1.3. Problema di soddisfacimento di vincoli

Un **problema di soddisfacimento di vincoli (CSP)** è definito da un insieme di variabili ognuna con un proprio *dominio* e un insieme di *vincoli*. Una sua **soluzione** è un'assegnazione totale che soddisfa tutti i vincoli.

Un *CSP finito* ha un numero finito di variabili di dominio finito. Oltre a metodi per CSP finiti, si prenderanno in considerazione anche algoritmi per casi con variabili dal dominio continuo. Molti esempi vengono dal mondo dei giochi, come il Sudoku o la Criptoaritmetica.

Esempio 16. (Robot consegne) Si può costruire un CSP definendo:

- le attività da svolgere: a, v, c, d, e e i *momenti* possibili: 1, 2, 3, 4;
- le rispettive variabili A, B, C, D, E , tutte con lo stesso dominio, ossia con $\text{dom}(A) = \text{dom}(B) = \text{dom}(C) = \text{dom}(D) = \text{dom}(E) = \{1, 2, 3, 4\}$;
- l'insieme dei *vincoli*:

$$\{B \neq 3; C \neq 2; A \neq B; B \neq C; C < D; A = D; E < A; E < B; E < C; E < D; B \neq D\}$$

Legati ai CSP, si possono definire diversi problemi di complessità crescente:

- determinare se esista una soluzione o meno;
- trovare una soluzione;
- contare il numero di soluzioni;
- enumerare tutte le soluzioni;
- trovare la soluzione migliore, rispetto a una data misura di qualità;
- determinare se alcuni enunciati siano veri per tutte le soluzioni.

I CSP possono risultare difficili per il loro carattere multidimensionale; il compito base è quello di trovare una soluzione. Già nel caso di CSP con domini finiti tale problema è NP-completo; metodi sistematici hanno una complessità esponenziale ma, ove possibile, si sfrutta la struttura dello spazio di ricerca.

3.2. Risoluzione di CSP tramite ricerca

Un algoritmo **Generate-And-Test** è un algoritmo *esaustivo* per CSP finiti semplice ma inefficiente. Per trovare una soluzione si generano e controllano in modo sistematico, una alla volta, le possibili assegnazioni totali; si restituirà la prima che soddisfa tutti i vincoli. Per trovare tutte le soluzioni si deve continuare a iterare, conservando le soluzioni via via trovate.

Si osservi che con n domini di cardinalità d si hanno d^n possibili assegnazioni totali; con e vincoli il numero totale di test è $O(ed^n)$: al crescere di n diventa rapidamente intrattabile, servono quindi soluzioni alternative.

3.2.1. Algoritmi su Grafo di Ricerca

In alternativa, si possono sfruttare *algoritmi di ricerca su grafo* per spazi di stati determinati dalle assegnazioni parziali. L'idea conduttrice è quella di testare i vincoli su assegnazioni parziali di ambito limitato crescente: se un'assegnazione parziale viola un vincolo, anche le assegnazioni totali che la estendono lo violeranno quindi è possibile potare lo spazio di ricerca.

Si può usare la DFS per cercare tutte le soluzioni di un CSP con variabili V_s e vincoli C_s , estendendo via via l'assegnazione parziale *contesto_parziale*, dove:

- V_s insieme delle variabili senza assegnazione in *contesto_parziale*;
- C_s insieme dei vincoli che coinvolgono almeno una variabile in V_s ;
- chiamata iniziale: `DFS_solver ( $V_s, C_s, \emptyset$ )`.

Rispetto a Generate-And-Test, i test anticipati consentono di potare sotto-alberi, risparmiando lavoro.

INSERIRE ALGORITMO

L'albero di ricerca risultante ha dimensioni che dipendono dall'ordine di scelta delle variabili. Un ordine *statico*, ad esempio sempre prima A , poi B , poi C , risulterà meno efficiente di uno *dinamico*. Ma l'ordine ottimale potrebbe essere più difficile da trovare.

3.3. Algoritmi basati su Consistenza

Si introduce una nozione di **consistenza** intesa come compatibilità, o coerenza logica, fra assegnazioni e vincoli. Una **rete di vincoli** (*constraint network*) indotta da un CSP è costituita da un *grafo bipartito*:

- un nodo (*circolare*) per ogni variabile, con un dizionario *dom* che, per ogni variabile X , contenga l'insieme $dom[X]$ di valori possibili, inizialmente impostato con l'intero suo dominio;
- un nodo (*rettangolare*) per ogni vincolo c ;
- un arco $\langle X, c \rangle$ per ogni variabile X nell'ambito del vincolo c .

Un arco $\langle X, c \rangle$ si dice **consistente** rispetto ai domini (*domain consistent*) se e solo se $\forall x \in \text{dom}[X] : \{X = x\}$ soddisfa c .

Dato il vincolo c su $\{X, Y_1, \dots, Y_k\}$, l'arco $\langle X, c \rangle$ è **consistente** se e solo se $\forall x \in \text{dom}[X], \exists y_1, \dots, y_k : y_i \in \text{dom}[Y_i]$ tale che $\{X = x, Y_1 = y_1, \dots, Y_k = y_k\}$ soddisfa c .

Una **rete consistente** contiene solo archi consistenti. Se $\langle X, c \rangle$ non è consistente allora per qualche valore di $\text{dom}[X]$ non ci sono valori di $Y_1, \dots, Y_k$ tali che l'assegnazione risultante soddisfa c , quindi *eliminando* tali valori da $\text{dom}[X]$ si può ripristinare la consistenza di $\langle X, c \rangle$. È bene notare, però, che l'eliminazione di valori da un dominio può rendere altri archi non consistenti.

L'idea di base è quella di rendere la rete consistente restringendo i domini. A tale scopo, si considera l'insieme *inconsistenti* degli archi potenzialmente non consistenti:

- si inizializza *inconsistenti* con tutti gli archi del grafo;
- si ripete fino a svuotare *inconsistenti*: estratto un arco $\langle X, c \rangle$ da *inconsistenti* se $\langle X, c \rangle$ non è consistente, si deve restringere $\text{dom}[X]$. Si devono, poi, aggiungere a *inconsistenti* gli archi resi non consistenti dal passo precedente: $\langle Z, c' \rangle$, $c' = c$, con ambito che comprende X e una diversa Z .

INSERIRE ALGORITMO.

GAC termina con una rete consistente con variabili dai domini *ridotti*. Tre casi sono possibili:

1. un dominio è vuoto quindi non ci sono soluzioni: se uno è vuoto, lo saranno anche altri domini ridotti ad esso connessi, già prima della terminazione;
2. i domini sono tutti ridotti a un solo valore, quindi la *soluzione è unica*;
3. altrimenti, si è ottenuto CSP semplificato cui applicare altri metodi.

Dei metodi alternativi sono algoritmi basati sulla **consistenza dei percorsi**.

3.4. Separazione dei domini

L'idea base degli algoritmi di **separazione dei domini** è quella di decomporre il CSP in una serie di *casi disgiunti* da risolvere separatamente; le soluzioni sono ricostruite riunendo quelle trovate per i diversi casi. Ad esempio:

- X binaria, dominio $\{t, f\}$, due problemi ridotti: trovare le soluzioni con $X = t$ e quelle con $X = f$; se ne basta una, secondo caso considerato solo se il primo non ha soluzione;
- A con dominio $\{1, 2, 3, 4\}$, ci sono diverse maniere per separare i valori:
 - un caso per ciascun valore; ciò fa fare più strada con una sola suddivisione alla volta;
 - due sottoinsiemi disgiunti: $A \in \{1, 2\}$ e $A \in \{3, 4\}$; questo taglia di più in meno passi.

Nel seguente schema di algoritmo si integra l'approccio basato sulla consistenza in un algoritmo ricorsivo:

1. si semplifica il CSP in input tramite `GAC()`
2. se non è risolto direttamente:
 1. si seleziona una variabile, con dominio almeno binario;
 2. si partiziona il dominio ottenendo (2+) problemi semplificati;
 3. si risolvono ricorsivamente tali problemi.

INSERIRE ALGORITMO

L'algoritmo fornisce tutte le soluzioni:

- se un dominio è vuoto non ci sono soluzioni;
- se il dominio ha un solo valore si ha una sola soluzione;
- ritornando dalla ricorsione restituisce l'unione delle soluzioni dei 2 casi;
- c'è modo di usare anche algoritmi di ricerca su grafo, ma qui contano le soluzioni e non i cammini.

Un possibile miglioramento è il seguente: se un'assegnazione rende il grafo *non connesso*, ogni componente può essere risolta separatamente. Si ricava facilmente una soluzione ricombinando le soluzioni delle componenti. Il conteggio del numero di soluzioni è fattibile in modo efficiente.

3.5. Eliminazione di Variabili

L'**eliminazione di variabili**, *variabile elimination (VE)*, è una tecnica che semplifica la rete dei vincoli rimuovendo variabili. L'idea di base è quella di eliminare progressivamente le variabili, una alla volta, ottenendo CSP semplificati da risolvere e infine, a partire dalle loro soluzioni, ricostruire quelle dei CSP più complessi: quando si elimina la variabile X si deve costruire un nuovo vincolo sulle rimanenti che rifletta gli effetti dei vincoli su X ; esso sostituisce tutti i vincoli su X producendo una rete semplificata; alla fine, ogni soluzione di un CSP ridotto va estesa per ottenere una soluzione del CSP che ricomprenda anche X .

Per eliminare una variabile X :

1. considerate le relazioni relative a tutti i vincoli su X , sia $r_X(X, \bar{Y})$ quella ottenuta dal join di tali relazioni, vincolo che codifica l'*influenza* di X su $\bar{Y}$, insieme delle altre variabili nell'ambito di r_X , vicine di X nel grafo dei vincoli;
2. la *proiezione* di r_X su $\bar{Y}$ sostituisce tutte le relazioni in cui occorre X ;
3. si ottiene un CSP ridotto, senza la X , da risolvere ricorsivamente. Al ritorno delle chiamate ricorsive si estendono le tabelle-soluzioni per il CSP ridotto tramite join con r_X , per aggiungere la colonna delle assegnazioni a X . Nel caso-base resta una sola variabile, per cui la soluzione da restituire è la tabella con i valori del dominio consistenti con i vincoli.

Esempio 17. Si consideri un CSP con le variabili A, B, C di dominio $\{1, 2, 3, 4\}$ e sia B la variabile da eliminare, inclusa nei vincoli: $A < B$ e $B < C$. Per eliminarla, si fa il join tra le relazioni dei vincoli su B :

<u>A</u>	<u>B</u>		<u>B</u>	<u>C</u>		<u>A</u>	<u>B</u>	<u>C</u>
1	2		1	2				
1	3		1	3		1	2	3
1	4	$\bowtie$	1	4	=	1	2	4
2	3		2	3		1	3	4
2	4		2	4		2	3	4
3	4		3	4				

La proiezione della tabella-join su A e C induce una nuova relazione senza la B :

<u>A</u>	<u>C</u>
1	3
1	4
2	4

Tale vincolo sostituisce gli originari e contiene tutte le informazioni utili al resto della rete; con VE poi si risolve il resto della rete semplificata.

Per avere una o tutte le soluzioni a partire dalla soluzione del CSP ridotto: si memorizza la relazione del join su A, B, C per estendere la soluzione della rete ridotta includendo B .

INSERIRE ALGORITMO

Seguono alcune osservazioni:

- *caos base*: rimane una sola variabile, quindi una soluzione esiste se ci sono righe nelle relazioni finali e saranno tutte relative a una sola variabile basterà intersecarle;
- caso *ricorsivo*: l'ordine di selezione delle variabili ha un impatto sull'efficienza; al ritorno se bastasse una soluzione, si restituisce solo una tupla di $R \bowtie S$. È garantito che sia parte d'una soluzione; se un valore di R non avesse tuple, non ci sarebbero soluzioni con tale valore.

VE può essere combinato con algoritmi basati su consistenza da usare per semplificare il problema quando si elimina una variabile. Le tabelle intermedie risulteranno più piccole.

3.6. Ricerca Locale

Nel caso di spazi molto grandi, o infiniti, non è pensabile una ricerca sistematica sull'intero spazio. Si può puntare su metodi *mediamente efficienti* per trovare soluzioni, ma *senza garanzie* di ritrovamento anche quando esistono. Perciò tali metodi sono utili quando si sa che, verosimilmente, ce ne sono.

La classe dei metodi di **ricerca locale**, comunemente investigati nell'ambito della *ricerca operativa* e dell'*AI*, comprende molte tecniche, con uno stesso schema-base:

Algoritmo 2: Schema della RICERCA LOCALE

```
1 procedure Local_search( $V_s$ , dom,  $C_s$ )
2 Input:
3    $V_s$ : insieme di variabili
4   dom: funzione che restituisce il dominio di una variabile
5    $C_s$ : insieme di vincoli da soddisfare
6 Output:
7   | assegnazione totale che soddisfa i vincoli
8 Local:
9   |  $A$  dizionario di valori indicizzato dalle variabili in  $V_s$ 
10 repeat {try}
11   for each  $X \in V_s$  do
12     |  $A[X] \leftarrow$  valore (casuale) da dom( $X$ )
13   {walk}
14   while not ferma_walk() and  $A$  non soddisfa  $C_s$  do
15     | Selezionare  $Y \in V_s$  e un valore  $w \in \text{dom}(Y)$ 
16     |  $A[Y] \leftarrow w$ 
17   if  $A$  soddisfa  $C_s$  then
18     | return  $A$ 
19 until terminazione
```

- si inizia con un'assegnazione totale di un valore a ciascuna variabile;
- si tenta di migliorare l'assegnazione iterativamente effettuando passi di *miglioramento*, passi *casuali* e *ripartenze* da assegnazioni iniziali differenti.

Ogni iterazione della **repeat** rappresenta un **tentativo** (*try*): con il primo ciclo **for each** si ha l'**inizializzazione casuale** di A ; per ogni assegnazione casuale successiva si ha una **ripartenza casuale** in alternativa anche congetture più informate basate su euristiche o conoscenza pregressa, poi migliorate iterando. Nel ciclo **while** si effettua una **ricerca locale** (*walk*) nello spazio delle assegnazioni:

- si seleziona una assegnazione tra i possibili **successori** di A che differiscono per il valore assegnato a una sola variabile;
- si ha lo stop se si è trovata una soluzione o si avvera il criterio di `ferma_walk()`, ad esempio è stato raggiunto un numero massimo di iterate.

La fermata non è garantita: l'algoritmo può divergere se il CSP non ha soluzione. In alcuni casi, anche se ne esistessero, potrebbe rimanere intrappolato in una regione; una garanzia di *completezza* dipende dai criteri di selezione e di stop.

Random Sampling è la versione della ricerca locale in cui:

- `ferma_walk()` risulta sempre vera quindi il ciclo `while` non viene mai eseguito: si continua indefinitamente a provare assegnazioni casuali che possano soddisfare tutti i vincoli;
- l'algoritmo è completo ossia garantisce di trovare la soluzione se questa esiste, tuttavia il tempo richiesto non può essere limitato e tipicamente risulta molto lento;
- la sua efficienza dipende dalle dimensioni dei domini e dal numero di soluzioni esistenti.

Random Walk è la versione della ricerca locale in cui:

- `ferma_walk()` risulta sempre falsa per cui non si hanno ripartenze casuali: si esce dal ciclo `while` solo se si trova una soluzione e, nel ciclo, si ripete la selezione casuale di una variabile e un valore da assegnarle;
- l'algoritmo è completo con passi più veloci rispetto al resampling di tutte le variabili, ma può richiedere più passi, in base alla distribuzione delle soluzioni;
- in alternativa, quando le dimensioni dei domini delle variabili differiscono, si può selezionare a caso una variabile e poi un valore del suo dominio oppure selezionare casualmente una coppia variabile-valore, il che favorisce la selezione di variabili con dominio più grande.

3.6.1. Miglioramento Iterativo

L'approccio del **miglioramento iterativo** è un tipo di ricerca locale che prevede la selezione del *miglior successore* vicino all'assegnazione corrente in termini di una **funzione obiettivo**. Quando la funzione è da minimizzare (una funzione di *costo* o di *perdita*), il metodo prende il nome di *Greedy Descent*, *discesa sbrigativa*. Se, invece, è da massimizzare, il metodo prende il nome di *Greedy Ascent*, o anche **Hill Climbing**. Nel seguito si considereranno solo funzioni da *minimizzare*, per l'altro criterio sarà sufficiente cambiare il segno della funzione. Nei casi di parità di valore della funzione si opera una scelta casuale.

Si distinguono:

- **ottimi locali**, ossia assegnazioni non migliorabili da alcun successore, come i *minimi* o *massimi locali* trovati, rispettivamente, con *greedy descent* o *ascent*;

- **ottimi globali**, quelli con valutazione massima fra tutte le assegnazioni, che risultano comunque anche ottimi locali.

Una funzione di valutazione tipica per i CSP è il numero di **conflitti**, come i vincoli violati: un'assegnazione totale con 0 conflitti è una soluzione. Tale funzione può essere raffinata pesando i vincoli in maniera differenziata. Adottando il *numero di conflitti*, si dirà che un CSP è *soddisfacibile* se si trova un minimo globale con valore nullo ovvero è non soddisfacibile se il minimo globale ha valore positivo: se si trova un minimo locale con valore positivo, non è detto che esso sia globale, e quindi che il CSP non sia soddisfacibile.

Si considera il miglior successore anche quando questo non ha una migliore valutazione rispetto all'assegnazione corrente. È possibile che l'algoritmo trovi ottimi locali che risultano successori reciproci e che continui a passare da uno all'altro senza poter trovare una soluzione. Questo rende l'algoritmo *incompleto*.

3.6.2. Algoritmi stocastici

Per evitare minimi locali che non siano anche globali, negli **algoritmi stocastici** si ricorre a una maggiore casualità. Le mosse casuali previste saranno:

1. il **random restart**, in cui valori scelti a caso per tutte le variabili: costituisce una mossa casuale globale che consente di ripartire da regione anche completamente diverse dello spazio;
2. il **random step**, una mossa casuale locale da alternare a passi di ottimizzazione: utilizzata nel *Greedy descent / ascent* permette pasasi in direzione opposta in modo da sfuggire a minimi / massimi locali.

Integrando *massimo miglioramento iterativo* e mosse *casuali* si definiscono algoritmi per la **ricerca locale stocastica**.

Esempio 18. (Spazi 2D) Si veda la seguente figura:

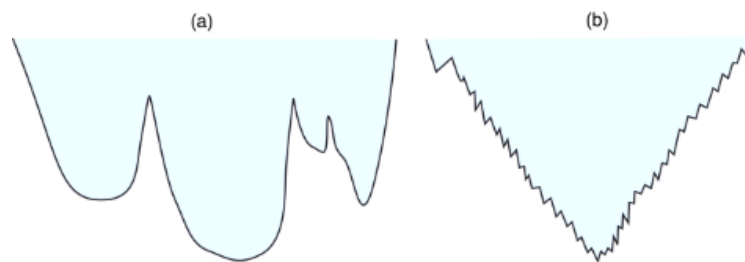


Figura 9: Due spazi di ricerca con caratteristiche diverse

Il *successore* si ottiene tramite un piccolo passo dall'attuale posizione verso sinistra o destra.

- spazio di ricerca (a): *Greedy descent* può trovare facilmente minimi locali, serve un *random restart* che porti nella parte centrale nella quale si converge rapidamente verso uno globale; invece *random walk* non funzionerebbe bene; richiederebbe molti piccoli passi casuali per uscire da un minimo locale;
- spazio di ricerca (b): *random restart* rimane bloccato a cercare tra numerosi minimi locale mentre *random walk* con *greedy descent* potrebbe evitare tali minimi locali.

Con spazi con diverse caratteristiche in regioni differenti si può pensare di ricorrere a un **catalogo di algoritmi** da cui scegliere.

3.7. Varianti della ricerca Locale

Molte altre varianti sono possibili nella *scelta del successore* e nelle scelte *casuali*. Con domini *limitati* tutti i valori possono essere scelti per i successori. Con domini più estesi si possono considerare solo alcuni valori, risparmiando tempo, in genere si considerano solo quelli più vicini ai precedenti, ma esistono metodi più sofisticati di selezione.

3.7.1. Tabu Search

La **tabu search** è una forma di ricerca locale con memoria che evita la modifica di assegnazioni introdotte di recente. Si memorizzano le variabili modificate negli ultimi t passi (**tenure**) che andranno considerate non selezionabili. In tal modo si evitano i cicli dopo poche assegnazioni. Va ottimizzato il parametro t . Nella sua implementazione, se è piccolo si può considerare una lista delle variabili modificate di recente, invece se è grande si memorizza per ogni variabile il passo in cui si è avuto l'ultima sua modifica.

3.7.2. Passo di Massimo Miglioramento

In questo metodo di ricerca locale, si seleziona una coppia variabile-valore che porta al miglioramento di valutazione massimale. In caso di più coppie si opera una scelta casuale.

Nella sua implementazione «ingenua» data l'assegnazione totale corrente, per ogni variabile X e ogni valore $v \in \text{dom}(X)$ diverso da quello corrente, si confronta l'assegnazione corrente con quella in cui si assegna $X = v$. Si seleziona quindi una delle coppie di *massimo miglioramento*. Si noti che la modifica potrebbe portare a *differenze negative* nella valutazione, ossia a peggioramenti. Inoltre, le variabili non coinvolte in vincoli possono essere trascurate.

In un'implementazione alternativa si adotta una coda con priorità fatta di coppie variabile-valore pesate. Per ogni X e ogni $v \in \text{dom}(X)$ non assegnato in A , in coda ci sarà una coppia $\langle X, v \rangle$ con peso $w = h(A') - h(A)$. Si considera allora il miglioramento dell'assegnazione A' ottenuta sostituendo $X = v$ rispetto ad A . Questo

dipenderà dai valori assegnati a X e dai suoi vicini nella rete dei vincoli, non da quelli assegnati alle altre variabili. A ogni iterata si seleziona una *coppia-successore* di massimo miglioramento, ossia con *peso minimale*. Ogni nuova assegnazione comporta il ricalcolo dei pesi e il riordimento della coda, ma solo per coppie con variabili presenti in vincoli il cui soddisfacimento è mutato.

3.7.3. Scelta a due fasi

L'algoritmo di **scelta a due fasi** prevede la selezione della coppia articolata in:

1. selezione della variabile per la riassegnazione;
2. selezione del valore.

Si gestisce una coda con priorità di variabili con associato un peso pari al numero dei conflitti in cui ognuna sia coinvolta. Ad ogni passo:

1. si seleziona X che partecipa a più conflitti;
2. le si cambia il valore assegnato, scegliendolo fra quelli che minimizzano il numero di conflitti oppure casualmente.

Vanno infine ricalcolati i pesi per le variabili coinvolte in vincoli il cui soddisfacimento è mutato.

3.7.4. Algoritmo Any-Conflict

L'idea-base dell'algoritmo **Any-Conflict** è quella di scegliere, per la modifica, una **variabile conflittuale** ossia una che partecipa a uno o più conflitti. A ogni iterata: si seleziona casualmente una variabile conflittuale, non necessariamente quella con più conflitti, quindi le si assegna, in alternativa, un valore che minimizzi il numero di conflitti oppure un valore casuale.

Sono possibili alcune varianti, ad esempio cambiando il criterio di selezione casuale della variabile: si può scegliere prima un conflitto e poi una variabile coinvolta oppure si opera una semplice scelta casuale di una variabile conflittuale. La differenza fra i due criteri sta nella probabilità di selezione di una variabile: nel primo essa dipende dal numero di conflitti in cui la variabile è coinvolta, nel secondo si ha la stessa probabilità per tutte le variabili.

3.7.5. Simulated Annealing

Il **simulated annealing**